



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
НИЖНИЙ НОВГОРОД

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ

Высшая Школа Экономики

Нижегородский филиал

Кафедра информационных систем и технологий

Л.В. Савченко, И.А. Макаров, А.В. Савченко

Машинное обучение и анализ табличных данных

Учебно-методическое пособие

Нижний Новгород

2026 г.

УДК 004.89

ББК 16.63

Рецензент:

Асеева Н.В.- к.ф-м.н., доцент, декан НИУ ВШЭ - Нижний Новгород

Савченко Л.В., Макаров И.А., Савченко А.В. Машинное обучение и анализ табличных данных: Учебно-методическое пособие. – М.: Постер-М, 2026. – 55 с.

ISBN 978-5-907929-35-7

Учебно-методическое пособие раскрывает методы решения основных задач машинного обучения (линейная регрессия, классификация, кластеризация) с помощью стандартных классификаторов из библиотеки `scikit-learn`, применение и оценку показателей качества при решении задач, акцент сделан на анализе практических примеров. Пособие предназначено для самостоятельной работы студентов при изучении курсов, связанных с машинным обучением по программе обучения бакалавров направления 38.03.05 «Бизнес-информатика», 09.03.04 «Программная инженерия», 01.03.02 «Прикладная математика и информатика».

© Савченко Л.В., Макаров И.А., Савченко А.В., 2026

Содержание

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ.....	4
ВВЕДЕНИЕ.....	5
1. Общие понятия машинного обучения.....	6
2. Задача регрессии.....	10
2.1. Постановка задачи.....	10
2.2. Методы решения задачи регрессии.....	12
2.3. Метрики качества.....	13
2.3.1. Примеры практического применения с использованием библиотеки scikit-learn.....	15
2.3.1. Практические рекомендации.....	25
2.4. Выводы.....	26
3. Задача классификации.....	27
3.1. Постановка задачи.....	27
3.2. Методы решения задачи классификации.....	28
3.3. Метрики качества бинарной классификации.....	31
3.4. Метрики качества множественной классификации.....	35
3.5. Примеры практического применения с использованием библиотеки scikit-learn.....	36
3.6. Выводы.....	44
4. Задача кластеризации.....	45
4.1. Постановка задачи.....	45
4.2. Методы решения задачи кластеризации.....	45
4.3. Метрики качества.....	47
4.4. Примеры практического применения.....	49
4.5. Выводы.....	52
Заключение.....	52
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	53
Приложение А. Задания для самостоятельного решения.....	54

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

MAE (Mean Absolute Error) – средняя абсолютная ошибка

MAPE (Mean Absolute Percentage Error) – средняя абсолютная процентная ошибка

MSE (Mean Square Error) – средняя квадратичная ошибка

RMSE (Root Mean Square Error) – корень средней квадратичной ошибки

R2 – Коэффициент детерминации

SGD (stochastic gradient descent) – стохастический градиентный спуск

ГО – глубокое обучение

ИИ – искусственный интеллект

МНК – метод наименьших квадратов

МО – машинное обучение

ВВЕДЕНИЕ

Машинное обучение (МО, machine learning) представляет собой современную и важную область искусственного интеллекта (ИИ, artificial intelligence), которая с каждым годом становится все более востребованной и разнообразной [1]. За последние несколько лет МО и ИИ получили новый толчок в развитии, вызванный огромным и постоянно растущим объемом данных и вычислительных мощностей [2]. Основной идеей МО является способность компьютерных систем «учиться» на основе данных и опыта, а не зависеть исключительно от явных программных инструкций. МО описывает способность компьютерных систем учиться на основе обучающих данных для конкретных задач, чтобы автоматизировать процесс построения аналитической модели и решения связанных задач [3, 4]. Глубокое обучение (ГО, deep learning) – это концепция машинного обучения, основанная на искусственных нейронных сетях. МО и ГО быстро стали мощными инструментами во многих областях, в том числе в распознавании изображений [5, 6] и речи [7, 8], обработке естественного языка [9, 10] и даже медицине [11].

В основе МО лежит идея создания алгоритмов и моделей, которые способны анализировать данные, выявлять закономерности и делать прогнозы. Вместо того чтобы жестко прописывать правила и условия, как это делается в классическом программировании, в МО система сама «находит» правила на основе большого объема информации. Это делает МО идеальным инструментом для работы с данными, которые слишком сложны или разнообразны, чтобы быть обработанными с помощью традиционных методов [2]. Машинное обучение можно разделить на несколько основных категорий:

1) Обучение с учителем (supervised learning): в этом подходе модель обучается на размеченных данных, где каждому входному примеру сопоставляется правильный ответ. Примеры задач включают *классификацию (classification)* – отнесение объекта к тому или иному классу объектов, и *регрессию (regression)* – предсказание некоторого значения на основе ограниченного количества примеров. Отличными примерами для данных задач являются: определение какое письмо является спамом, а какое полезной информацией; определение по МРТ снимку возможных заболеваний; распознавание видов животных; распознавание рукописных цифр; определение эмоции пользователя по звуковому сигналу; прогнозирование цены акций на фондовом рынке или цены на недвижимость и т.п. [3].

2) Обучение без учителя (unsupervised learning): в этом случае модель пытается выявить структуры или закономерности в данных без предоставления правильных ответов. Примерами являются задачи кластеризации (группировка схожих объектов) и понижения размерности данных (сжатие информации). Примеры задач: определение человеческой активности (активный образ жизни, неактивный образ жизни); выделение пользовательских

групп по определенным предпочтениям (маркетинг); группировка стран по уровню развития и т.п. [3].

3) Обучение с подкреплением (reinforcement learning): здесь модель обучается, взаимодействуя с окружающей средой и получая награды или штрафы за свои действия. Этот подход активно применяется в области управления роботами, игровой индустрии, беспилотных видов транспорта [3].

МО также находит широкое применение в современном мире, от рекомендательных систем на платформах потокового видео до автономных автомобилей и диагностики заболеваний. Эта область продолжает развиваться, и ее потенциал практически не ограничен, что делает ее одной из самых важных областей компьютерных наук [1].

1. Общие понятия машинного обучения

- **Модель (Model):** модель в контексте машинного обучения – это математическое или статистическое представление процесса, которое позволяет делать прогнозы или выявлять закономерности в данных. Модель может быть функцией или алгоритмом, который принимает на вход набор признаков (входных данных) и выдает на выходе результат, который может быть числовым значением, классом или другими формами данных. Например, линейная регрессия – это модель, которая представляет собой линейную функцию для прогнозирования числовых значений на основе входных признаков.

- **Данные (Data):** это информация, которую модель использует для обучения и прогнозирования. Данные могут включать в себя признаки (features) и целевую переменную (target), если речь идет о задаче обучения с учителем.

- **Параметры модели (Model parameters):** параметры модели – это настраиваемые переменные, которые определяют ее форму и поведение. Параметры модели используются для адаптации модели к данным в процессе обучения. В случае линейной регрессии параметры модели включают в себя коэффициенты (веса), учитываемые для каждого признака, и смещение (intercept), который определяет базовое значение прогноза. В машинном обучении задача обучения модели заключается в нахождении наилучших значений параметров, чтобы минимизировать ошибку прогноза.

- **Целевая переменная (Target Variable):** целевая переменная – это значение, которое модель пытается предсказать. В задачах регрессии – это числовое значение, в задачах классификации – метка класса.

- **Градиентный спуск (Gradient descent):** градиентный спуск – это оптимизационный алгоритм, используемый для настройки параметров модели так, чтобы минимизировать функцию потерь (функцию, которая измеряет разницу между прогнозами модели и истинными значениями целевой переменной). Основная идея заключается в том, что

алгоритм итеративно корректирует параметры модели, двигаясь в направлении, противоположном градиенту функции потерь, чтобы найти минимум ошибки.

- Стандартный градиентный спуск: на каждой итерации алгоритм вычисляет градиент функции потерь по отношению к параметрам модели и обновляет параметры с некоторым шагом (скоростью обучения).
- Стохастический градиентный спуск (SGD, stochastic gradient descent): В этой вариации градиентного спуска на каждой итерации выбирается случайный поднабор данных (mini-batch) для вычисления градиента. Это позволяет ускорить обучение на больших наборах данных и сделать алгоритм более стохастическим.
- Методы с оптимизацией скорости обучения: некоторые вариации градиентного спуска, такие как Adam, Adagrad и RMSprop, используют адаптивные методы для изменения скорости обучения на лету в зависимости от градиента. Это позволяет быстрее сходиться к оптимальным значениям параметров.

- **Обучение (Training):** этот процесс включает в себя подачу обучающих данных модели с целью настройки ее параметров. Модель «учится» на данных, чтобы делать более точные прогнозы.

- **Функция потерь (Loss):** это функция, которая измеряет разницу между прогнозами модели и истинными значениями целевой переменной. Для задачи регрессии часто используются функции потерь, такие как средняя квадратичная ошибка (MSE, Mean Squared Error) или средняя абсолютная ошибка (MAE, Mean Absolute Error).

- **Тестирование (Testing):** после обучения модели ее производительность оценивается на отдельном тестовом наборе данных. Это позволяет определить, насколько хорошо модель обобщает знания на новые данные.

- **Переобучение (Overfitting):** Переобучение происходит, когда модель слишком сложная и пытается адаптироваться к обучающим данным до такой степени, что начинает захватывать шум или случайные колебания в данных, вместо выявления реальных закономерностей. Это приводит к тому, что модель демонстрирует отличные результаты на обучающем наборе данных, но плохо обобщается на новые данные.

Признаки переобучения включают:

- Высокую точность на обучающих данных и низкую на тестовых или новых данных.
- Слишком сложные формы зависимости в модели, которые не подтверждаются действительными данными.
- Маленькие коэффициенты регуляризации или отсутствие регуляризации в модели.

Для борьбы с переобучением можно использовать следующие методы:

- Увеличение объема данных для обучения.

- Использование более простых моделей с меньшим количеством параметров.
- Применение регуляризации, такой как L1 или L2 регуляризация, чтобы уменьшить веса параметров.
- Использование кросс-валидации для оценки производительности модели на независимых данных.

- **Недообучение (Underfitting):** недообучение, наоборот, происходит, когда модель слишком проста, чтобы захватить сложности данных. В этом случае модель не способна уловить реальные зависимости в данных и, следовательно, демонстрирует низкую точность как на обучающих, так и на тестовых данных.

Признаки недообучения включают:

- Низкую точность как на обучающих, так и на тестовых данных.
- Простые формы зависимости в модели, которые не могут хорошо аппроксимировать сложные закономерности данных.
- Маленькие или нулевые веса параметров.

Для борьбы с недообучением можно использовать следующие методы:

- Использование более сложных моделей с большим количеством параметров.
- Добавление новых признаков или их полиномиальных комбинаций.
- Увеличение количества эпох обучения (в случае нейронных сетей) или глубины деревьев (в случае деревьев решений).
- Использование кросс-валидации для настройки гиперпараметров модели.

На рисунке 1 приведен пример переобучения и недообучения моделей в задаче предсказания цены дома в зависимости от его размера.

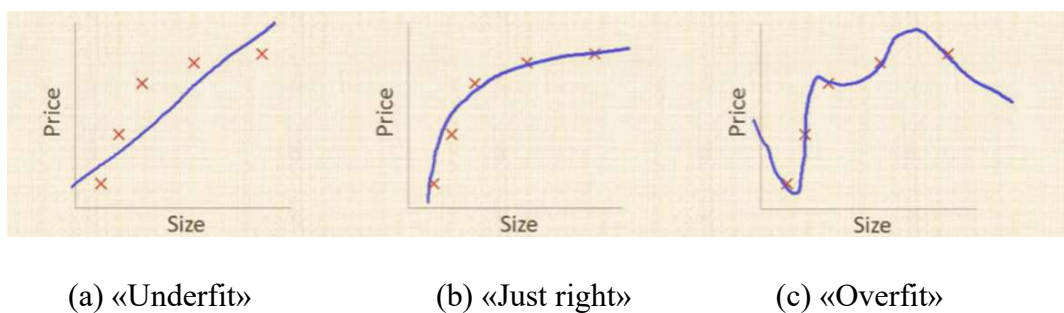


Рис.1. Пример переобучения и недообучения моделей

Идеальным решением является достижение баланса между недообучением и переобучением, что называется «подгонкой» (fitting) модели к данным. Это требует тщательной настройки модели и мониторинга ее производительности на отдельном

тестовом наборе данных, чтобы убедиться, что модель хорошо обобщается на новые данные.

- **Гиперпараметры (Hyperparameters)**: это параметры модели, которые не настраиваются в процессе обучения, а устанавливаются до начала обучения. Примеры гиперпараметров включают в себя скорость обучения, глубину деревьев, коэффициент регуляризации и другие.

- **Валидация (Validation)**: это процесс оценки производительности модели на отдельном наборе данных (валидационном наборе), который не использовался в процессе обучения. Это помогает настраивать гиперпараметры и предотвращать переобучение.

- **Кросс-валидация (Cross-Validation)**: это метод оценки производительности модели, при котором данные разбиваются на несколько подмножеств (фолдов, folds), и модель обучается и оценивается несколько раз, чтобы получить более устойчивую оценку производительности.

- **Регуляризация (Regularization)**: это метод контроля переобучения путем добавления дополнительных ограничений на параметры модели. Это может включать в себя L1 и L2 регуляризацию.

2. Задача регрессии

2.1. Постановка задачи

Задача регрессии в машинном обучении является одной из фундаментальных задач и используется для прогнозирования числовых значений на основе имеющихся данных [12]. Она находит широкое применение во многих областях, включая финансы, экономику, науку о данных, медицину и многие другие.

В задаче регрессии у нас имеется набор данных, состоящий из пар (X, y) , где X – это входные признаки (или фичи), а y – числовое значение, которое мы хотим предсказать. Наша цель состоит в том, чтобы построить математическую модель, которая будет определять функциональную зависимость между входными данными и выходными значениями.

Математически задачу регрессии можно выразить следующим образом:

$$y = f(X) + \varepsilon, \quad (1)$$

где y – целевая переменная, которую мы хотим предсказать, X – вектор входных признаков, который описывает объект, $f(X)$ – неизвестная функция, которая представляет собой зависимость между X и y , ε – случайная ошибка, которая представляет собой шум или невоспроизводимую составляющую данных.

Для решения задачи в этой постановке необходимо:

- Найти наилучшую аппроксимацию неизвестной функции $f(X)$ на основе имеющихся данных.
- Оценить качество модели с использованием соответствующих метрик качества.

Примеры задач регрессии могут быть очень разнообразными:

1) Прогнозирование цен на недвижимость на основе характеристик (площадь, год постройки, количество комнат, место положение и т.п.).

2) Оценка финансовых показателей компании, таких как выручка или прибыль, на основе исторических данных.

3) Предсказание потребительского спроса на товары и услуги.

4) Прогноз страховых выплат на основе состояния здоровья работника (возраст, индекс массы тела, показатели давления, наличие вредных привычек и т.п.).

При разработке модели регрессии существует несколько ключевых шагов и аспектов:

- 1) Подготовка данных: прежде чем начать решение задачи регрессии, необходимо очистить и предобработать данные. Это может включать в себя удаление выбросов, заполнение пропущенных значений, масштабирование признаков и кодирование категориальных переменных.

- 2) Выбор модели: один из самых важных шагов в решении задачи регрессии – выбор подходящей модели. Это может быть линейная регрессия, полиномиальная регрессия, деревья решений или более сложные модели, такие как случайный лес или нейронные сети. Выбор зависит от природы данных и ожидаемой сложности зависимости между признаками и целевой переменной.
- 3) Обучение модели: после выбора модели необходимо провести обучение. Этот шаг включает в себя настройку параметров модели так, чтобы она могла наилучшим образом аппроксимировать зависимость между входными данными X и выходными значениями y . Обучение модели может быть итеративным процессом, где модель постепенно улучшает свои предсказательные способности.
- 4) Оценка модели: после обучения модели необходимо оценить ее качество. Для задачи регрессии существуют различные метрики качества, такие как средняя абсолютная ошибка (MAE), средняя квадратичная ошибка (MSE), коэффициент детерминации (R^2) и другие. Оценка позволяет определить, насколько хорошо модель справляется с предсказанием целевых значений.
- 5) Интерпретация результатов: понимание, какие признаки оказывают наибольшее влияние на предсказания модели, может быть важным аспектом задачи регрессии. Это позволяет выявить ключевые факторы, влияющие на целевую переменную, и принимать более обоснованные бизнес-решения.
- 6) Настройка и оптимизация: в случае необходимости можно провести настройку и оптимизацию модели, изменяя ее параметры или выбирая другой алгоритм. Это может потребовать проведения кросс-валидации для выбора оптимальных гиперпараметров.

Задача регрессии является важным инструментом для прогнозирования числовых значений в различных областях. Благодаря широкому спектру доступных методов и библиотек, таких как `scikit-learn` [13] в Python, исследователи и инженеры могут разрабатывать точные и устойчивые модели регрессии для решения разнообразных задач прогнозирования [14]. Пример вопроса на задачу регрессии с независимого экзамена по анализу данных приведен на рисунке 2. При прогнозировании количества ручек, купленных за год, мы имеем дело с задачей регрессии, т.е. нам нужно дать прогноз на год (оценить количество ручек), целевая переменная (y) – то, что нужно спрогнозировать, т.е. количество ручек.

Аналитики некоторой компании проводят исследование спроса клиентов на шариковые ручки.
Данные представлены в виде таблицы с полями "год покупки ручек", "возраст", "пол", "уровень образования", "средний доход", "город проживания клиентов".
Аналитики пытаются спрогнозировать количество шариковых ручек, которое каждый клиент купил за 2018, 2019, 2020, 2021, 2022 год.
Выберите два верных утверждения.

- ☒ Целевой переменной в данной задаче является количество купленных ручек
- ☐ Решается задача классификации
- ☐ Целевой переменной в данной задаче является клиент
- ☒ Решается задача регрессии
- ☐ Решается задача обучения без учителя

Рис.2. Пример задачи с независимого экзамена по анализу данных

2.2. Методы решения задачи регрессии

В библиотеке `scikit-learn` существует несколько стандартных классификаторов для решения задачи линейной регрессии:

- Линейная регрессия (`LinearRegression()`¹) - соответствует линейной модели

$$y = w_0 + w_1 \cdot x_1 + \dots + w_n \cdot x_n \quad (2)$$

с коэффициентами $w = (w_0, \dots, w_n)$, модель минимизирует остаточную сумму квадратов между наблюдаемыми целями в наборе данных и целями, предсказанными с помощью линейного приближения.

- Линейная модель с L1 регуляризацией (`Lasso`²).

$$Lasso = \sum_{i=1}^N (y_n - \hat{y}_n)^2 + \alpha \|w\|_1, \quad (3)$$

здесь $\alpha \geq 0$ – параметр регуляризации, когда $\alpha = 0$, то модель эквивалентна обычному методу наименьших квадратов. Когда $\alpha = 0$, то рекомендуется использовать модель `LinearRegression()`.

- Линейная модель с L2 регуляризацией (`Ridge`³).

$$Ridge = \sum_{i=1}^N (y_n - \hat{y}_n)^2 + \alpha \|w\|_2^2, \quad (4)$$

здесь $\alpha \geq 0$ – параметр регуляризации.

¹ https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

² https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html

³ https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html

L1-регуляризация способствует разреженности функции, когда лишь немногие признаки не равны нулю. L2-регуляризация способствует появлению малых весовых коэффициентов модели, но не способствует их точному равенству нулю.

- Метод наименьших квадратов (МНК).

$$\vec{W} = (X^T \cdot X + \alpha \cdot I)^{-1} \cdot X^T \cdot \vec{y} \quad (5)$$

МНК помогает найти неизвестные коэффициенты $w_0, w_1, \dots, w_n$ в уравнении. Здесь X – матрица признаков, в которой самый левый столбец состоит из 1. T – означает транспонированную матрицу, -1 – обратную матрицу, $\alpha \geq 0$ – параметр регуляризации, I – единичная матрица.

2.3. Метрики качества

Метрики качества – это численные показатели, которые используются для оценки производительности модели машинного обучения. Они предоставляют количественную информацию о том, насколько хорошо модель справляется с задачей обучения, и позволяют сравнивать различные модели или настройки. Выбор подходящей метрики зависит от типа задачи, характера данных и целей исследования. Давайте рассмотрим некоторые из наиболее распространенных метрик качества в контексте задачи регрессии.

1) Средняя квадратичная ошибка (MSE - Mean Square Error): средняя квадратичная ошибка измеряет средний квадрат разницы между прогнозами модели и фактическими значениями целевой переменной. Вычисляется следующим образом:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (6)$$

Здесь n – количество примеров в тестовом наборе данных, y_i – фактическое значение целевой переменной для i -го примера, а $\hat{y}_i$ – прогноз модели для того же примера. Чем меньше значение MSE , тем ближе прогнозы модели к реальным данным.

Недостатком использования MSE является то, что если на одном или нескольких неудачных примерах, возможно, содержащих аномальные значения будет допущена значительная ошибка, то возведение в квадрат приведёт к ложному выводу, что вся модель работает плохо. С другой стороны, если модель даст небольшие ошибки на большом числе примеров, то может возникнуть обратный эффект – недооценка модели.

2) Корень из средней квадратической ошибки (RMSE – Root Mean Square Error): вычисляется как квадратный корень из MSE :

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (7)$$

MSE и $RMSE$ могут быть минимизированы с помощью одного и того же функционала, поскольку квадратный корень является неубывающей функцией. Например,

если у нас есть два набора результатов работы моделей А и В, и MSE для А больше, чем MSE для В, то мы можем быть уверены, что $RMSE$ для А больше $RMSE$ для В. Справедливо и обратное: если $MSE(A) < MSE(B)$, то и $RMSE(A) < RMSE(B)$. Отметим, что ошибка $RMSE$ измеряется в тех же единицах, что и целевая переменная y .

Влияние каждой ошибки на $RMSE$ пропорционально величине квадрата ошибки. Поэтому большие ошибки оказывают непропорционально большое влияние на $RMSE$. Следовательно, $RMSE$ можно считать чувствительной к аномальным значениям.

3) Средняя абсолютная ошибка (MAE - Mean Absolute Error): средняя абсолютная ошибка измеряет средний абсолютный модуль разницы между прогнозами и фактическими значениями. MAE вычисляется следующим образом:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (8)$$

MAE более устойчива к выбросам в данных, чем MSE , так как она не учитывает квадраты разниц.

4) Коэффициент детерминации (R^2): коэффициент детерминации измеряет долю объясненной дисперсии в общей дисперсии данных. Он позволяет оценить, какую часть изменчивости целевой переменной модель объясняет.

R^2 принимает значения от 0 до 1, где 0 означает, что модель не объясняет вариабельность данных, а 1 – что она объясняет всю вариабельность.

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (\bar{y} - y_i)^2} \quad (9)$$

Практически, в числителе данного выражения стоит среднеквадратическая ошибка оцениваемой модели, а в знаменателе модели присутствует только константа.

Главным преимуществом коэффициента детерминации является его инвариантность к масштабу данных. Кроме того, он всегда изменяется в диапазоне от $-\infty$ до 1. При этом значения близкие к 1 указывают на высокую степень соответствия модели данным. Очевидно, что это имеет место, когда отношение в формуле стремится к 0, т.е. ошибка модели с переменными намного меньше ошибки модели с константой.

$R^2=0$ показывает, что между независимой и зависимой переменными модели имеет место функциональная зависимость.

Когда значение коэффициента близко к 0 (т.е. ошибка модели с переменными примерно равна ошибке модели только с константой), это указывает на низкое соответствие модели данным, когда модель с переменными работает не лучше модели с константой.

Кроме этого, бывают ситуации, когда коэффициент R^2 принимает отрицательные значения (обычно небольшие). Это произойдёт, если ошибка модели среднего становится меньше ошибки модели с переменной. В этом случае оказывается, что добавление в модель с константой некоторой переменной только ухудшает её (т.е. регрессионная модель с переменной работает хуже, чем предсказание с помощью простой средней).

На практике используют следующую шкалу оценок. Модель, для которой $R^2 > 0.5$, является удовлетворительной. Если $R^2 > 0.8$, то модель рассматривается как очень хорошая. Значения, меньшие 0.5 говорят о том, что модель плохая.

5) Средняя абсолютная процентная ошибка (MAPE - Mean Absolute Percentage Error) вычисляется следующим образом:

$$MAPE = \frac{100}{n} \sum_{i=1}^n \frac{|y_i - \hat{y}_i|}{|y_i|} \quad (10)$$

Эта ошибка не имеет размерности и очень проста в интерпретации. Её можно выражать как в долях, так и в процентах. Если получилось, например, что MAPE=10.5, то это говорит о том, что ошибка составила 10.5% от фактического значения.

На рисунке 3 приведен пример с независимого экзамена на выбор метрик качества. Перед нами задача прогнозирования (предсказать число просмотров статьи в зависимости от ее характеристик), т.е. регрессии, поэтому нужно выбрать одну из метрик качества для этой задачи (MSE, RMSE, MAE, MAPE, R^2).

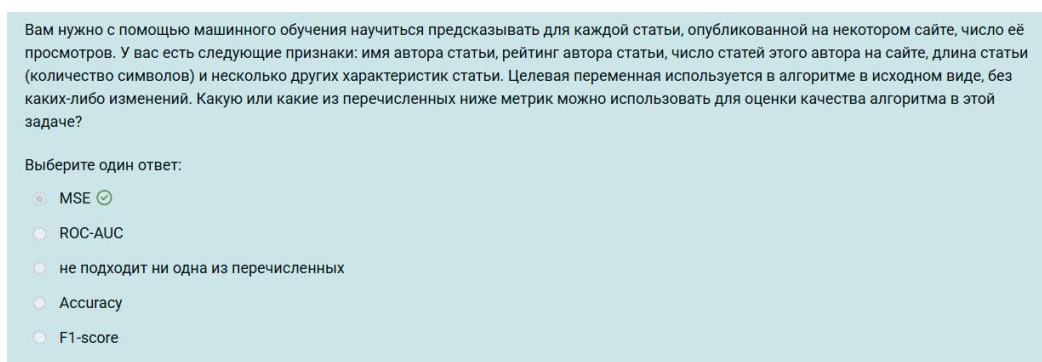


Рис.3. Пример задания с независимого экзамена по анализу данных

2.3 Примеры практического применения с использованием библиотеки `scikit-learn`

Решим несколько задач регрессии с использованием библиотеки `scikit-learn`.

Пример 1

Рассмотрим решение задачи предсказания цены на жилье в зависимости от площади и количества спален. С набором данных можно ознакомиться по ссылке [ex1data2.txt](#)⁴.

В первом столбце указана площадь дома (в квадратных футах), во втором – количество спален, в третьем – цена дома. Таким образом, X_1 – площадь дома, X_2 – количество спален в доме, y – цена (целевая функция, т.е. то, что необходимо спрогнозировать). Алгоритм построения модели линейной регрессии представлен ниже.

Алгоритм 1 – Алгоритм решения задачи прогнозирования цены дома

Входные данные: X_1 – площадь дома, X_2 – количество спален в доме, y – цена дома

Выходные данные: метрики качества MSE, MAE, R^2

1: Подготовить набор данных к работе и разбить его на множества train и test

$X_{\text{train}}, X_{\text{test}}, y_{\text{train}}, y_{\text{test}} = \text{train_test_split}(X, y)$

2: Обучить модель линейной регрессии для задачи предсказания цены на дом на множестве train

$\text{model_LR} = \text{LinearRegression}()$

$\text{model_LR.fit}(X_{\text{train}}, y_{\text{train}})$

3: Вычислить метрики качества на множестве test

$\text{predictions_LR_test} = \text{model_LR.predict}(X_{\text{test}})$

$\text{mse_LR_test} = \text{mean_squared_error}(y_{\text{test}}, \text{predictions_LR_test})$

$\text{mae_LR_test} = \text{mean_absolute_error}(y_{\text{test}}, \text{predictions_LR_test})$

$\text{r2_LR_test} = \text{r2_score}(y_{\text{test}}, \text{predictions_LR_test})$

Далее рассмотрим подробное решение этой задачи с помощью методов библиотеки `scikit-learn` [6]. С кодом можно ознакомиться по ссылке [Linear_Regression_1.ipynb](#)⁵.

Поэтапное решение задачи:

1. Загрузка данных:

```
data = np.loadtxt(path, delimiter=',')
```

Здесь мы используем функцию `numpy.loadtxt()`, чтобы загрузить данные из файла `path="/content/gdrive/MyDrive/ex1data2.txt"`, разделенные запятыми. Загруженные данные сохраняются в переменной `data`.

2. Разделение данных на признаки (фичи) и целевую переменную:

⁴ <https://drive.google.com/file/d/1KNfwVfCre9yMqTbgeN72nttCF1VBxRp2/view?usp=sharing>

⁵ https://drive.google.com/file/d/1o5cbKOMj9_Lq1pVcwZdWhewDB_Np0Gyd/view?usp=sharing

```
X = data[:, :-1]
y = data[:, -1]
```

Мы используем индексацию массивов NumPy, чтобы разделить данные на фичи X и целевую переменную y . X содержит все столбцы кроме последнего, а y содержит только последний столбец.

3. Нормализация данных:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Здесь мы создаем объект **StandardScaler()**, который будет использоваться для нормализации данных. Затем мы вызываем метод **fit_transform()** для нормализации фичей X . Нормализованные данные сохраняются в переменной X_scaled . Нормализация данных является важным этапом в предобработке данных, который помогает привести различные единицы измерения к единому виду. Это облегчает сравнение, анализ и обработку данных, а также улучшает работу алгоритмов машинного обучения

4. Разделение данных на обучающую и тестовую выборки:

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=2)
```

Мы используем функцию **train_test_split()** из модуля `sklearn.model_selection` для разделения нормализованных фичей X_scaled и целевой переменной y на обучающую и тестовую выборки. Здесь мы указываем, что размер тестовой выборки составляет 20% от общего размера данных, а `random_state=2` гарантирует воспроизводимость разделения.

5. Создание модели линейной регрессии и обучение на обучающих данных:

```
model_LR = LinearRegression()
model_LR.fit(X_train, y_train)
```

Мы создаем объект модели **LinearRegression()** и вызываем метод **fit()** для обучения модели на обучающих данных X_train и y_train .

6. Задание степени полинома и создание полиномиальных фичей:

```
poly_degree = 2 # Задайте степень полинома
poly_features = PolynomialFeatures(degree=poly_degree)
X_poly_train = poly_features.fit_transform(X_train)
X_poly_test = poly_features.fit_transform(X_test)
```

Мы задаем степень полинома равной 2 и создаем объект **PolynomialFeatures** с указанной степенью. Затем мы преобразуем обучающие данные X_train и тестовые данные X_test в полиномиальные фичи с помощью метода **fit_transform()**.

7. Создание модели полиномиальной регрессии и обучение на обучающих данных с полиномиальными фичами:

```
model_PLR = LinearRegression()
model_PLR.fit(X_poly_train, y_train)
```

Мы создаем объект модели **LinearRegression()** и вызываем метод **fit()** для обучения модели на обучающих данных с полиномиальными фичами **X_poly_train** и **y_train**.

8. Получение предсказаний на тестовых данных для модели линейной регрессии:

```
predictions_LR_test = model_LR.predict(X_test)
```

Мы используем метод **predict()** для получения предсказаний модели линейной регрессии на тестовых данных **X_test**. Предсказания сохраняются в переменной **predictions_LR_test**.

9. Получение предсказаний на тестовых данных для модели полиномиальной регрессии:

```
predictions_LR_test = model_LR.predict(X_test)
predictions_PLR_test = model_PLR.predict(X_poly_test)
```

Мы используем метод **predict()** для получения предсказаний модели полиномиальной регрессии на тестовых данных с полиномиальными фичами **X_poly_test**. Предсказания сохраняются в переменной **predictions_PLR_test**.

10. Вычисление метрик для оценки качества моделей:

```
# Оценка моделей на тестовой выборке
rmse_LR_test = np.sqrt(mean_squared_error(y_test, predictions_LR_test))
mae_LR_test = mean_absolute_error(y_test, predictions_LR_test)
r2_LR_test = r2_score(y_test, predictions_LR_test)
rmse_PLR_test = np.sqrt(mean_squared_error(y_test, predictions_PLR_test))
mae_PLR_test = mean_absolute_error(y_test, predictions_PLR_test)
r2_PLR_test = r2_score(y_test, predictions_PLR_test)
```

Мы используем функции из модуля **sklearn.metrics** для вычисления метрик сравнения предсказанных значений с истинными целевыми значениями. **mean_squared_error()** вычисляет среднеквадратическую ошибку (MSE), **mean_absolute_error()** вычисляет среднюю абсолютную ошибку (MAE), а **r2_score()** вычисляет R-квадрат (коэффициент детерминации).

11. Вывод результатов на экран:

```
print(f"Mean Squared Error (Регрессия) на тестовой выборке: {rmse_LR_test}")
print(f"Mean Absolute Error (Регрессия) на тестовой выборке: {mae_LR_test}")
print(f"r2 score (Регрессия) на тестовой выборке: {r2_LR_test}")
print(f"Mean Squared Error (Полиномиальная регрессия) на тестовой выборке: {rmse_PLR_test}")
print(f"Mean Absolute Error (Полиномиальная регрессия) на тестовой выборке: {mae_PLR_test}")
print(f"r2 score (Полиномиальная регрессия) на тестовой выборке: {r2_PLR_test}")
```

Мы выводим значения метрик на тестовой выборке для модели линейной регрессии (Линейная регрессия) и модели полиномиальной регрессии (Полиномиальная регрессия) с использованием форматированной строки и переменных.

12. Получение предсказаний для моделей линейной и полиномиальной регрессии на тестовой выборке:

```
predictions_LR_test = model_LR.predict(X_test)
predictions_PLR_test = model_PLR.predict(X_poly_test)
```

Мы используем метод **predict()** для получения предсказаний моделей линейной и полиномиальной регрессии на тестовой выборке.

13. Визуализация результатов сравнения исходных и предсказанных значений на графиках:

```
predictions_LR_test = model_LR.predict(X_test)
predictions_PLR_test = model_PLR.predict(X_poly_test)

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.scatter(y_test, predictions_LR_test, color='blue')
plt.title('True Values vs. Predicted Values (Linear Regression)')
plt.xlabel('True Values')
plt.ylabel('Predicted Values')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--', color='red')

plt.subplot(1, 2, 2)
plt.scatter(y_test, predictions_PLR_test, color='green')
plt.title('True Values vs. Predicted Values (Polynomial Linear Regression)')
plt.xlabel('True Values')
plt.ylabel('Predicted Values')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], linestyle='--', color='red')

plt.tight_layout()
plt.show()
```

Мы создаем графики, чтобы визуально сравнить исходные и предсказанные значения для линейной и полиномиальной регрессии. Графики позволяют оценить качество моделей.

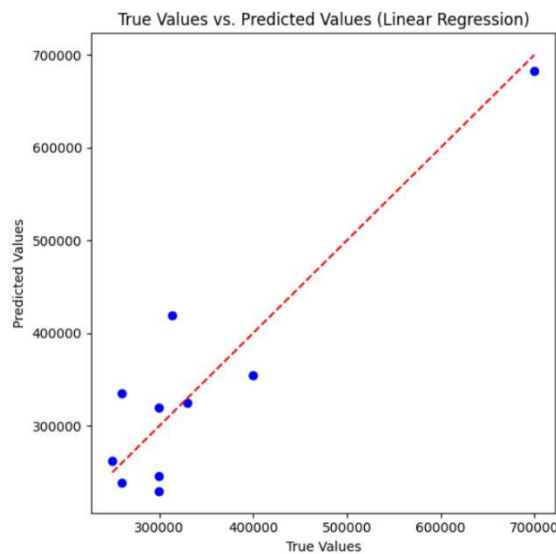


Рис.4. Построение модели линейной регрессии

На рисунке красной линией представлена построенная (predict) модель линейной регрессии, а синими точками – истинные значения.

14. Создание и обучение моделей Ridge и Lasso регрессии с настройкой параметра регуляризации alpha:

```
# Ridge регрессия с L2-регуляризацией
alpha = 1.0 # Параметр регуляризации, можно настроить
ridge_model = Ridge(alpha=alpha)
ridge_model.fit(X_train, y_train)

# Lasso регрессия с L1-регуляризацией
alpha = 1.0 # Параметр регуляризации, можно настроить
lasso_model = Lasso(alpha=alpha)
lasso_model.fit(X_train, y_train)
```

Мы создаем модели Ridge и Lasso регрессии, задавая параметр регуляризации `alpha`, и обучаем их на обучающих данных.

15. Получение предсказаний для моделей Ridge и Lasso регрессии на тестовой выборке:

```
# Предсказания с регуляризацией
predictions_ridge_test = ridge_model.predict(X_test)
predictions_lasso_test = lasso_model.predict(X_test)
```

Мы используем метод `predict()` для получения предсказаний моделей Ridge и Lasso регрессии на тестовой выборке.

16. Вычисление метрик для оценки качества моделей Ridge и Lasso регрессии на тестовой выборке:

```
# Оценка моделей с регуляризацией
rmse_ridge_test = np.sqrt(mean_squared_error(y_test, predictions_ridge_test))
mae_ridge_test = mean_absolute_error(y_test, predictions_ridge_test)
r2_ridge_test = r2_score(y_test, predictions_ridge_test)
rmse_lasso_test = np.sqrt(mean_squared_error(y_test, predictions_lasso_test))
mae_lasso_test = mean_absolute_error(y_test, predictions_lasso_test)
r2_lasso_test = r2_score(y_test, predictions_lasso_test)
```

Мы используем функции из модуля **sklearn.metrics** для вычисления метрик оценки качества моделей. `mean_squared_error()` вычисляет среднеквадратическую ошибку (MSE), `mean_absolute_error()` вычисляет среднюю абсолютную ошибку (MAE), а `r2_score()` вычисляет R-квадрат (коэффициент детерминации).

17. Вывод результатов оценки моделей Ridge и Lasso регрессии на тестовой выборке:

```
print(f"Mean Squared Error (Ridge регрессия) на тестовой выборке: {rmse_ridge_test}")
print(f"Mean Absolute Error (Ridge регрессия) на тестовой выборке: {mae_ridge_test}")
print(f"r2 score (Ridge регрессия) на тестовой выборке: {r2_ridge_test}")
print(f"Mean Squared Error (Lasso регрессия) на тестовой выборке: {rmse_lasso_test}")
print(f"Mean Absolute Error (Lasso регрессия) на тестовой выборке: {mae_lasso_test}")
print(f"r2 score (Lasso регрессия) на тестовой выборке: {r2_lasso_test}")
```

Мы выводим значения метрик для оценки качества моделей Ridge и Lasso регрессии на тестовой выборке.

С использованием регуляризации ошибки для линейной регрессии Ridge() равны: `rmse=52676.50`, `mae=43105.53`, `r2=0.83`. Если посмотреть на данные, то среднее значение `y` `np.mean(y)=340412.65`, а ошибки `rmse` и `mae` чуть больше 10%. Для решения задачи

регрессии следует выбрать ту модель, которая имеет наименьшие ошибки RMSE и MAE и больший коэффициент R^2 .

Пример 2

Рассмотрим еще одну задачу регрессии – прогноз страховых выплат. В наборе данных по ссылке [insurance.csv](#)⁶ представлен размер выплаты страховой суммы (*charges* – целевая переменная) в зависимости от различных шести признаков (возраст, пол, индекс массы тела, количества детей, курит ли человек, возраст и регион проживания). Первые 9 строк набора данных представлены на рисунке 5.

age	sex	bmi	children	smoker	region	charges
19	female	27.9	0	yes	southwest	16884.924
18	male	33.77	1	no	southeast	1725.5523
28	male	33	3	no	southeast	4449.462
33	male	22.705	0	no	northwest	21984.47061
32	male	28.88	0	no	northwest	3866.8552
31	female	25.74	0	no	southeast	3756.6216
46	female	33.44	1	no	southeast	8240.5896
37	female	27.74	3	no	northwest	7281.5056
37	male	29.83	2	no	northeast	6406.4107

Рис. 5. Первые девять строк набора данных *insurance.csv*

Стоит обратить внимание на наличие категориальных признаков, т.е. нечисловых. Например, пол (male, female), курит ли человек (no, yes) и регион проживания (southwest, southeast, northwest, northeast). Для корректного обучения модели эти признаки должны быть преобразованы в числовые с помощью, например, алгоритма one-hot-encoding. На рисунке 6 представлен пример этого алгоритма при преобразовании трех категориальных признаков в числовые.

Type			
AA			
AB			
CD			

Onehot encoding →

Type	AA_Onehot	AB_Onehot	CD_Onehot
AA	1	0	0
AB	0	1	0
CD	0	0	1

Рис.6. Пример работы алгоритма one-hot-encoding

С кодом можно ознакомиться по ссылке [Linear_Regression_2.ipynb](#)⁷.

⁶ <https://drive.google.com/file/d/1-lgVk9AEem138I5VoufVXjbMYr9wnvZq/view?usp=sharing>

⁷ <https://drive.google.com/file/d/14JITbZiNXk5sahajmSNOBPylbpJ22GGk/view?usp=sharing>

Рассмотрим ниже основные блоки.

1. Загружаем наш датасет с помощью метода **read_csv()**.

```
# Загрузка данных
data = pd.read_csv(path)
```

2. Выбираем данные, которые имеют категориальный тип.

```
non_numeric_columns = data.select_dtypes(exclude=['number']).columns
```

3. С помощью метода **get_dummies()** меняет категориальные данные на числовые. Отметим, что **pd.get_dummies()** в библиотеке pandas создаёт новые бинарные столбцы для каждой уникальной категории, где 1 обозначает наличие категории, а 0 – отсутствие.

```
data = pd.get_dummies(data, columns = non_numeric_columns, drop_first=True)
```

4. Разделяем данные на X (признаки) и y (целевая переменная).

```
X = data.drop(columns=['charges'])
y = data['charges']
```

5. Разделяем данные на множества train и test.

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=234324)
```

6. Создаем модель линейной регрессии с помощью **LinearRegression()** и обучаем модель на данных из множества train.

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
```

7. Предсказываем значение целевой переменной на множестве test с помощью метода **predict()**.

```
y_pred = lr_model.predict(X_test)
```

8. Проводим оценку модели с помощью показателей качества

```
rmse = mean_squared_error(y_test, y_pred, squared=False)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

Значения показателей качества для данной модели равны $RMSE = 5441.26$, $MAE = 4038.26$, $R_2 = 0.81$. Отметим, что $R^2 > 0.8$, т.е. модель можно рассматривать как очень хорошую.

Пример 3

Рассмотрим решение предыдущей задачи с помощью метода МНК.

С кодом `Linear_Regression_2_МНК.ipynb`⁸.

Загрузка и обработка данных аналогична предыдущему примеру. Метод МНК реализуется по формуле в блоке:

⁸ https://drive.google.com/file/d/1okbB_MoHdIfThbC3sR4AHO1jpXAjexwl/view?usp=sharing

```
#МНК W= [(X^T·X+α·I)] ^(-1)·X^T·y
a=10
x_1 = np.array(y_train**0) # единичный столбец
x_1 = x_1.reshape(-1,1)
X_mnk = np.hstack((x_1, X_train))
print(np.float64(X_mnk))
X_train_T=X_mnk.transpose()
#print(X_train_T.shape,X_train.shape)
W=np.linalg.inv( (np.float64(X_train_T)).dot( np.float64(X_mnk)) +
a*(np.float64(np.eye(X_mnk.shape[1])))).dot(np.float64(X_train_T)).dot(np.float64(y_train)) #formula MNK

X_test_mnk=np.hstack((np.array(y_test**0).reshape(-1,1), X_test))
y_predict_mnk=X_test_mnk.dot(W)
```

```
[[ 1.   21.   22.135 ...  0.    0.    0.   ]
 [ 1.   37.   30.875 ...  1.    0.    0.   ]
 [ 1.   58.   33.1   ...  0.    0.    1.   ]
 ...
 [ 1.   61.   43.4   ...  0.    0.    1.   ]
 [ 1.   51.   25.8   ...  0.    0.    1.   ]
 [ 1.   58.   32.395 ...  0.    0.    0.   ]]
```

Здесь a – коэффициент регуляризации. Следует обратить внимание, что левый столбец матрицы X в методе МНК единичный. Это происходит, когда модель линейной регрессии в

формуле (2) мы переписываем в матричном виде: $\vec{y} = \begin{bmatrix} w_0 \\ w_1 \\ \dots \\ w_n \end{bmatrix} \times \begin{bmatrix} x_0 & x_1 & \dots & x_n \end{bmatrix}$. Из формулы

(2) видно, что при коэффициенте w_0 нет x , поэтому будем считать, что x входит в степени 0, т.е. равен 1. Значение показателей качества чуть хуже, чем для стандартных классификатор из предыдущего примера. Без нормализации данных значения равны $rmse= 6236.94$, $mae=4281.37$, $r_2=0.74$.

Пример 4

Рассмотрим решение задачи предсказания прибыли магазина в зависимости от количества людей, проживающих в городе с помощью фреймворка LAMA. LAMA (LightAutoML) – open source фреймворк автоматизированного машинного обучения. Задуман как инструмент, позволяющий прогрессировать в машинном обучении от начального уровня до эксперта. Более подробно с LAMA можно ознакомиться по ссылке <https://github.com/sb-ai-lab/LightAutoML?tab=readme-ov-file>.

С набором данных можно ознакомиться по ссылке data1.txt⁹.

В первом столбце указано количество население города (количество человек * 10.000), во втором – прибыль магазина (в \$10.000). Таким образом, X_1 – население города, y – прибыль (целевая функция, т.е. то, что необходимо спрогнозировать).

⁹ https://drive.google.com/file/d/1u7ONz4ww_XLKDYhum5dyQiHxvcsXYZ-s/view?usp=sharing

С кодом линейная_регрессия_LAMA.ipynb¹⁰.

Рассмотрим основные шаги:

1. Устанавливаем библиотеку LAMA

```
✓ 23 [2] !pip install -U lightautoml  
сек.
```

2. Определяем основные метрики качества для задачи регрессии

```
✓ 0 [7] from sklearn import metrics  
сек.  
  
def calc_metrics(true, pred):  
    stats={  
        'RMSE': round(np.sqrt(metrics.mean_squared_error(true, pred)), 4),  
        'MAE': round(metrics.mean_absolute_error(true, pred),4),  
        'R2': round(metrics.r2_score(true, pred),4)  
    }  
    print(f"RMSE: {stats['RMSE']}\nMAE: {stats['MAE']}\nR2: {stats['R2']}")  
    return stats
```

2. Считываем данные и создаем датафрейм, т.к. LAMA работает именно с ними.

```
data = pd.read_csv("/content/gdrive/MyDrive/Colab Notebooks/ML/data1.txt", header=None, sep=',')  
X = data.iloc[:, 0].values.reshape(-1, 1)  
Y = data.iloc[:, 1].values
```

```
df = pd.DataFrame({'X': X.reshape(-1), 'Y': Y.reshape(-1)})  
df.head()
```

	X	Y
0	6.1101	17.5920
1	5.5277	9.1302
2	8.5186	13.6620

3. Импортируем LAMA и устанавливаем параметры

```
[11] from lightautoml.automl.presets.tabular_presets import TabularAutoML  
from lightautoml.tasks import Task  
  
N_THREADS = 4  
N_FOLDS = 5  
RANDOM_STATE = 42  
TEST_SIZE = 0.3  
TIMEOUT = 300  
TARGET_NAME = 'Y'
```

4. Определяем задачу – регрессия и разбиваем данные на обучающую и тестовую выборки.

¹⁰ <https://drive.google.com/file/d/1snX57m8xLveK-xKOidKy6-LETBpf-803/view?usp=sharing>

```

task = Task('reg')

roles = {
    'target': TARGET_NAME,
}

train_data, test_data = train_test_split(
    df,
    test_size=TEST_SIZE,
    #stratify=df[TARGET_NAME],
    random_state=RANDOM_STATE
)

print(f'Data is splitted. Parts sizes: train_data = {train_data.shape}, test_data = {test_data.shape}')

train_data.head()

```

5. Обучаем модель

```

✓ 0 [13] automl = TabularAutoML(
  сек.    task = task,
          timeout = TIMEOUT,
          cpu_limit = N_THREADS,
          reader_params = {'n_jobs': N_THREADS, 'cv': N_FOLDS, 'random_state': RANDOM_STATE},
          )

✓ 1 [14] %%time
  мин.    out_of_fold_predictions = automl.fit_predict(train_data, roles = roles, verbose = 1)

```

6. Предсказываем целевую переменную и выводим метрики качества.

```

✓ 0 [15] test_predictions = automl.predict(test_data)
  сек.

✓ 0 calc_metrics(test_data[TARGET_NAME].values, test_predictions.data[:, 0])
  сек.

⇒ RMSE: 3.2859
  MAE: 2.1056
  R2: 0.6076
  {'RMSE': 3.2859, 'MAE': 2.1056, 'R2': 0.6076}

```

2.3.1. Практические рекомендации

- При работе с данными необходимо все переменные привести к числовому виду (алгоритм one-hot-encoding).

- Если признаки (X) имеют разную размерность, например, X_1 – количество этажей, X_2 – материал, X_3 – год постройки, X_4 – площадь, то целесообразно для улучшения показателей качества данные преобразовать к одному виду. Например, стандартизировать их или отнормировать ¹¹.

- Если признаков X много, то возможно, стоит проанализировать их и выбрать наиболее значимые.

¹¹ <https://scikit-learn.ru/6-3-preprocessing-data/>

- Стоит решить задачу, используя разные модели (Lasso(), Ridge(), LinearRegression(), МНК, LAMA), также стоит подобрать параметр регуляризации.

Опишем алгоритм решения задачи регрессии.

1. Считываем данные. Формируем признаки X и целевую переменную Y.
2. Разбиваем исходное множество на train и test (train_test_split()). На множестве train будем обучать модель, на множестве test будем считать показатели качества.
3. Определяем модель (Lasso, Ridge, МНК, LAMA).
4. Обучаем модель (fit()) на множествах X_train, Y_train.
5. Предсказываем (predict()) значения целевой переменной на множестве X_test. Получаем значения Y_predict, полученные моделью.
6. Вычисляем показатели качества (RMSE, MAE, R2) для Y_predict и Y_test
7. Делаем выводы о том, какая модель для решения задачи регрессии подходит лучше (лучше та модель, у которой лучше показатели качества, т.е. меньше ошибки RMSE, MAE и выше показатель R2).

2.4. Выводы

В данном разделе была рассмотрена задача линейной регрессии и методы ее решения (метод наименьших квадратов, стандартные классификаторы, LAMA).

Регрессия – это тип задач машинного обучения, где цель состоит в предсказании непрерывной целевой переменной (например, цены дома или продолжительности жизни) на основе некоторых признаков, модель линейной регрессии аппроксимирует зависимость между признаками и целевой величиной.

В разделе были введены показатели качества задачи регрессии (RMSE, MAE, R2). Также были рассмотрены примеры, демонстрирующие применения описанных подходов на практике, с полным кодом всех примеров можно ознакомиться по ссылкам в разделе или по QR кодам.

Этот раздел может быть полезен при решении задач регрессии, например, при построении модели предсказания цен на недвижимость, автомобилей, авиабилетов и т.п., а также при подготовке к независимому экзамену по анализу данных.

3. Задача классификации

3.1. Постановка задачи

Классификация – это разделение множества объектов на группы, называемые классами, на основе анализа их формального описания. При классификации каждая единица наблюдения относится к определенной группе или номинальной категории на основе некоторого качественного свойства [5]. Классификация может быть бинарной (есть два класса) и множественной (более двух классов). Примеры бинарной классификации: определение наличия болезни по МРТ снимку (болен/не болен), определение транзакции (подозрительная/не подозрительная), определение является ли email спамом или не является, определение кота или собаки на изображении и т.п. Примеры множественной классификации: распознавание рукописных цифр, распознавание эмоций человека (sad, angry, happy, neutral), распознавание животных и т.п [15-17].

Постановка задачи

Пусть X – множество описаний объектов, например, числовые значения признаков объектов, Y – конечное множество номеров (имен, меток) классов, которым могут принадлежать объекты. Существует неизвестная целевая зависимость – отображение $y^*: X \rightarrow Y$, которая представляет собой неизвестное отображение, которое сопоставляет каждому объекту из множества X номер (имя, метку) класса из множества Y . Значения зависимости известны только на объектах конечной обучающей выборки (множество train) $X_m = (x_1, y_1), \dots, (x_m, y_m)$, где x_i – объект из множества X , а y_i – соответствующий ему класс из множества Y , а $i \in [1, m]$. Требуется построить алгоритм $a: X \rightarrow Y$, способный классифицировать произвольный объект $x \in X$, то есть при подаче на вход какого-то объекта x получить номер m (имя, метку) класса из множества Y , которому этот объект соответствует. Также в зависимости от постановки задачи, классифицировать один и тот же предмет можно по-разному. Например, цель классификации изображений может состоять в том, чтобы присвоить изображению одну или несколько меток. Это может быть либо классификация с одной меткой (изображение может быть только в одной категории, исключая другие), либо классификация с несколькими метками (пометить необходимо все категории, к которым принадлежит изображение). Пример классификации с одной и несколькими метками изображен на рисунке 7 [18].

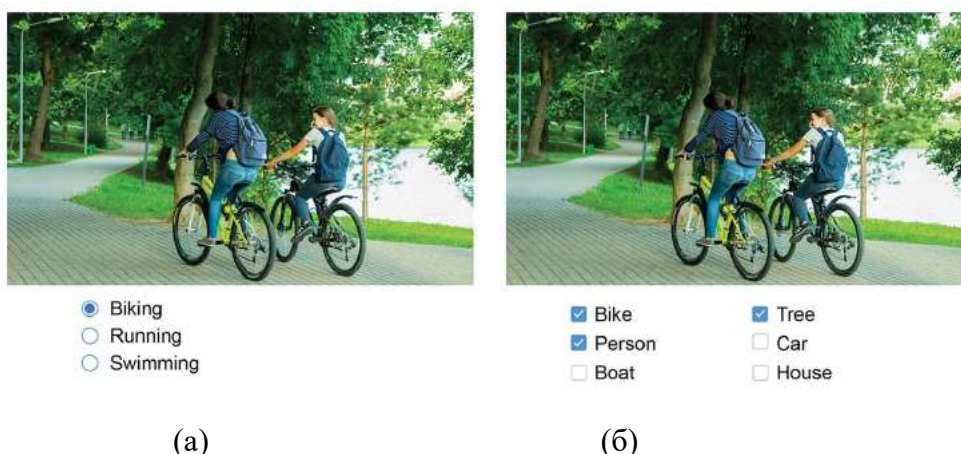


Рис.7. Пример классификации (а) – с одной меткой, (б) – с несколькими метками

3.2. Методы решения задачи классификации

Бинарная или двоичная классификация (binary classification) – это задача классификации элементов заданного множества на две группы/категории/класса (предсказание, какой из групп принадлежит каждый элемент множества) на основе правила классификации.

Методы, обычно используемые для двоичной классификации:

- Деревья решений (Decision Tree)

Цель состоит в том, чтобы создать модель, которая предсказывает значение целевой переменной, изучая простые правила принятия решений, выведенные из характеристик данных. Дерево можно рассматривать как кусочно-постоянное приближение. В основе метода лежит алгоритм разбиения данных на части с использованием различных критериев.

- Случайные леса (Random Forest)

Основная идея заключается в использовании большого ансамбля решающих деревьев, каждое из которых отдельно имеет низкое качество классификации, но за счёт количества в совокупности получается хорошим.

Основные параметры метода в библиотеке scikit-learn:

- `n_estimators` – число «деревьев» в «случайном лесу», default=100.
- `max_features` – число признаков для выбора расщепления. При построении каждого дерева, в каждом узле для выбора лучшего расщепления выбирается не всё множество признаков, а только случайное подмножество размера `max_features`.
- `max_depth` – определяет максимальное количество уровней (узлов) между корнем дерева и самым дальним листом.

- `min_samples_split` – минимальное число объектов, необходимое для того, чтобы узел дерева мог бы расщепиться, `default=2`.
- `min_samples_leaf` – минимальное число объектов в листьях, `default=1`.

Иллюстрация метода представлена на рисунке 8.

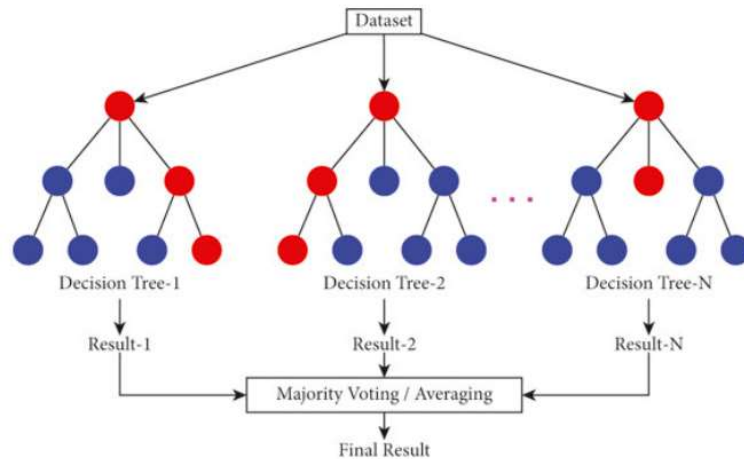


Рис. 8. Иллюстрация метода Random Forest

- Методы опорных векторов (Support Vector Classification)

Основная идея метода заключается в построении гиперплоскости (пространства, размерность которого на единицу меньше размерности исходного), разделяющей объекты выборки оптимальным образом. Иллюстрация метода опорных векторов представлена на рисунке ниже.

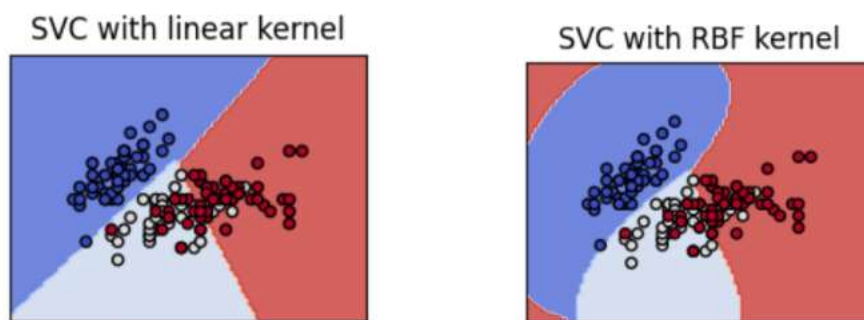


Рис. 9. Иллюстрация метода опорных векторов с линейной функцией ядра и с rbf (Radial Basis Function) функцией

- Градиентный спуск (Stochastic Gradient Descent)

Метод оптимизации в задачах классификации. Он минимизирует функцию потерь, обновляя веса модели на каждой итерации на основе градиента функции потерь. Это позволяет алгоритму градиентного спуска находить локальные минимумы функции потерь и настраивать модель для оптимальной классификации.

- Логистическая регрессия (Logistic Regression)

Частный случай обобщенной линейной регрессии, в котором предполагается, что зависимая переменная принимает два значения и имеет биномиальное распределение. В отличие от линейной регрессии, логистическая предсказывает вероятность принадлежности к классу. Она преобразует линейную комбинацию признаков в вероятность.

- Байесовские методы (Gaussian Naive Bayes)

Методы основаны на теореме Байеса – которая утверждает, что если плотности распределения каждого из классов известны, то искомый алгоритм можно выписать в явном аналитическом виде – позволяющем вычислить вероятность принадлежности объекта к определенному классу на основе его признаков [19]. Метод Гаусса предполагает, что признаки независимы и каждый класс распределен по Гауссовскому (нормальному) закону, на основе параметров которого вычисляется вероятность принадлежности.

- Градиентный бустинг (Gradient Boosting)

Градиентный бустинг [20] – метод, который последовательно строит ансамбль моделей, минимизируя функцию потерь с помощью градиентного подхода. Идея метода состоит в том, что мы сначала строим простую модель (обычно деревья решений). Затем смотрим, где эта модель ошиблась и добавляем новую модель, которая устраняет ошибки предыдущих моделей, этот шаг повторяет много раз. В итоге получаем сильную композицию слабых моделей. Основные параметры метода:

n_estimators – число деревьев

learning_rate (η) – скорость обучения

max_depth – глубина деревьев

min_samples_leaf – минимальное число объектов в листе.

Основные методы классификации в библиотеке scikit-learn, с подробным их описанием (реализацией и параметрами) можно ознакомиться по ссылкам:

- DecisionTreeClassifier¹²;

- RandomForestClassifier¹³;

- SVC¹⁴;

- SGDClassifier¹⁵;

¹² <https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

¹³ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html>

¹⁴ <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

¹⁵ https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.SGDClassifier.html

- LogisticRegression¹⁶;
- GaussianNB¹⁷;
- GradientBoostingClassifier¹⁸.

3.3. Метрики качества бинарной классификации

Перед тем, как описать метрики, необходимо ввести показатели качества или матрицу ошибок (Таблица 1). Рассмотрим возможные ошибки на примере бинарной классификации (спам/не спам).

Таблица 1 – Матрица ошибок бинарной классификации на примере определения спама

	Фактически положительные	Фактически отрицательные
Принято решение «Спам» (положительные примеры, $y=1$)	TP (true positive) – верно классифицированные положительные примеры (спам распознан как спам).	FP (false positive) – ложное обнаружение (ошибка I рода), т.е. не спам распознан как спам.
Принято решение «не спам» (отрицательные примеры, $y=0$)	FN (false negative) – положительные примеры, которые классифицированы как отрицательные (ошибка II рода), т.е. спам распознан как не спам.	TN (true negative) – верно классифицированные отрицательные примеры (не спам распознан как не спам).

Здесь $TP+FN=100\%$ и $TN+FP=100\%$.

Метрика Accuracy

Accuracy – это показатель, который описывает общую точность предсказания модели по всем классам. Это особенно полезно, когда каждый класс одинаково важен. Он рассчитывается как отношение количества правильных прогнозов ($TP+TN$) к их общему количеству. В scikit-learn для подсчета данной метрики используется метод `accuracy_score()`¹⁹.

Стоит учесть, что метрика accuracy может быть обманчивой. Один из таких случаев – это несбалансированные данные. Предположим, у нас есть всего 600 единиц данных, из которых 550 относятся к классу Positive и только 50 – к классу Negative. Поскольку

¹⁶ https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

¹⁷ https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.GaussianNB.html

¹⁸ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingClassifier.html>

¹⁹ https://scikit-learn.org/stable/modules/generated/sklearn.metrics.accuracy_score.html

большинство данных принадлежит к одному классу, ассигасу для этого класса будет выше, чем для другого.

$$Accuracy(y, y^{pred}) = \frac{1}{N} \sum_{i=1}^N I[y_i = f(x_i)], \quad (11)$$

где N – количество примеров в тестовом множестве, y – истинное значение класса, y^{pred} – предсказанное значение класса, I – оператор который равен 1, если значение аргументов равно и 0 в противном случае.

Рассмотрим пример задания с независимого экзамена на применение метрик в задаче классификации.

Цель банка - найти мошенников при совершении банковских операций (мошеннических транзакций на несколько порядков меньше, чем не мошеннических). Пусть при обучении классификации для выявления мошеннических транзакций в банке ассигасу на тренировочных данных получилось равным 0.9, а на тестовых - 0.85. О чем может говорить эта ситуация?

Для решения задачи использовали логистическую регрессию с Lasso-регуляризацией.

Выберите наиболее подходящий вариант ответа.

- ☐ Модель переобучилась, так как качество на тесте ниже, чем на трейне.
- ☐ Модель обучилась как надо (нет ни недообучения, ни переобучения)
- ☐ Модель недообучилась, так как качество на трейне низкое
- ☒ Неизвестно, переобучилась или недообучилась модель, так как выбрана плохая метрика для данной задачи

Очистить мой выбор

Рис.10. Пример задания с независимого экзамена по анализу данных

В этой задаче данные несбалансированные, т.е. количество примеров для каждого класса разное (отличается на несколько порядков), поэтому метрика ассигасу не подходит для этой задачи. В данном случае лучше использовать метрики, которые описываются в следующем разделе (например, метрику F1).

Метрики Precision и Recall, F1-мера

Precision (точность) можно интерпретировать как долю объектов, названных классификатором положительными (TP+FP) и при этом действительно являющимися положительными (TP). В scikit-learn для подсчета данной метрики используется метод `precision_score()`²⁰.

$$Precision = \frac{TP}{TP + FP} \quad (12)$$

²⁰ https://scikit-learn.org/stable/modules/generated/sklearn.metrics.precision_score.html

Метрика Recall (полнота) показывает, какую долю объектов положительного класса (TP) из всех объектов положительного класса (TP+FN) нашел алгоритм. В scikit-learn для подсчета данной метрики используется метод `recall_score`²¹.

$$Recall = \frac{TP}{TP + FN} \quad (13)$$

F1-мера представляет собой баланс между точностью и полнотой, а если точнее – их среднее гармоническое и вычисляется по формуле:

$$F_1 = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}} = 2 \frac{Recall \cdot Precision}{Recall + Precision} = \frac{TP}{TP + \frac{FP + FN}{2}} \quad (14)$$

Значение F1-меры находится в диапазоне от 0 до 1, где 1 – это идеальное значение, указывающее на идеальное соотношение между точностью и полнотой. В scikit-learn для подсчета данной метрики используется метод `f1_score()`²².

Метрика AUC

ROC (receiver operating characteristic) AUC (area under the curve) – площадь под ROC-кривой – часто используют для оценивания качества упорядочивания алгоритмом объектов двух классов, значение лежит на отрезке $[0, 1]$.

Идеальному случаю соответствует ROC-кривая, проходящая через точку (0, 1), площадь под ней равна 1. Наихудшему – ROC-кривая, проходящая через точку (1, 0), площадь под ней – 0. Случайному – что-то похожее на диагональ квадрата, площадь примерно равна 0.5. Пример вопроса с независимого экзамена представлен на рисунке 9. Здесь имеет место случайное распределение, т.е. диагональ квадрата.

В выборке 600 объектов положительного класса и 400 объектов отрицательного класса.

Алгоритм бинарной классификации на всех объектах выборки предсказывает одну и ту же вероятность попасть в положительный класс, то есть

$a(x) = \beta$, где β - некоторое число из отрезка $[0; 1]$

Чему равен ROC-AUC алгоритма, посчитанный на этой выборке? Ответ округлите до десятых.

Ответ:

Рис.11. Пример задания с независимого экзамена

Другими словами, ROC-кривая показывает отношение между истинно положительными (TP+FN) и ложно положительными (TN+FP) предсказаниями при различных пороговых значениях классификации. AUC представляет собой площадь под ROC-кривой и даёт общую оценку производительности модели: чем выше показатель AUC,

²¹ https://scikit-learn.org/stable/modules/generated/sklearn.metrics.recall_score.html

²² https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

тем качественнее классификатор. В scikit-learn для подсчета данной метрики используется метод `roc_auc_score`²³. Рассмотрим еще один пример с независимого экзамена.

Алгоритм бинарной классификации для каждого объекта x_i выдает оценку b_i его принадлежности к положительному классу. Ниже в таблице даны предсказания b_i и правильные ответы y_i .

x_i	y_i
0.7	+1
0.6	-1
0.3	-1
0.45	+1
0.92	-1

Вычислите ROC-AUC. Ответ округлите до сотых.

Ответ: **ROC-AUC=0.5**

Рис.12. Пример задания с независимого экзамена

Здесь два положительных объекта: 0.7, 0.45 для них метка +1 и три отрицательных объекта: 0.6, 0.3, 0.92 для них метка 0 (или -1). Произведем подсчет метрики с помощью метода `roc_auc_score()`:

```
from sklearn.metrics import roc_auc_score
roc_auc_score([1,0,0,1,0],[0.7,0.6,0.3,0.45,0.92])
np.float64(0.5)
```

Также попробуем построить для этой задачи ROC-кривую, фиксируя пороги:

Порог 1: FP = 0, TP=0, т.е. таких данных нет.

Порог 0.9: FP = 1:3=0.33, TP=0, т.е. выше порога попал один отрицательный пример (0.92)

Порог 0.7: FP = 0.33, TP=1:2=0.5, т.е. выше порога попал один отрицательный пример (0.92, метка класса «-1») и один положительный пример (0.7, метка класса «+1»).

Порог 0.6: FP = 2:3=0.66, TP=0.5

Порог 0.4: FP = 0.66, TP=2:2=1

Порог 0.3: FP = 1, TP=1.

Напомним, что $TP = \frac{TP}{TP + FN}$ и $FP = \frac{FP}{TN + FP}$. Построим ROC-кривую.

²³ https://scikit-learn.org/0.16/modules/generated/sklearn.metrics.roc_auc_score.html

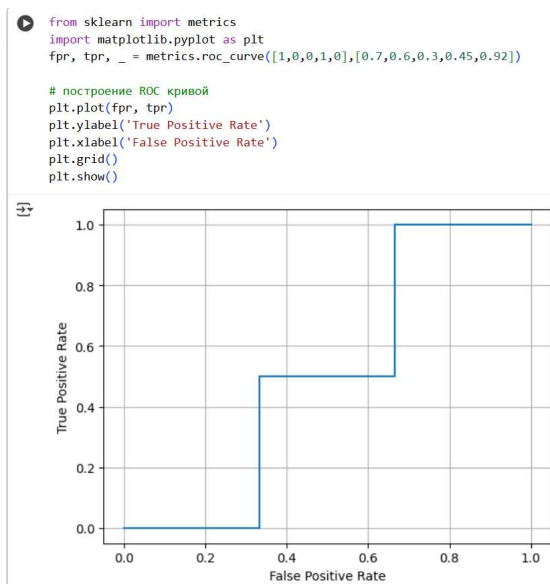


Рис. 13. Пример построения ROC-кривой

3.4. Метрики качества множественной классификации

Множественная классификация (multiply classification) – это задача классификации элементов заданного множества на более чем две группы/категории/класса (предсказание, какой из групп принадлежит каждый элемент множества) на основе правила классификации. Примеры задач классификации: распознавание рукописного текста; распознавание эмоций; классификация еды или животных.

Методы, обычно используемые для двоичной классификации:

- Деревья решений (DecisionTreeClassifier)
- Случайные леса (RandomForestClassifier)
- Методы опорных векторов (SVC)
- Градиентный спуск (SGDClassifier)
- Байесовские методы (GaussianNB)
- Градиентный бустинг (GradientBoostingClassifier)

Классификаторы и метрики качества по сравнению с бинарной классификацией не сильно меняются, однако требуется усреднение по классам для получения одной оценки для многоклассовой классификации.

Scikit-learn предоставляет несколько методов усреднения: макро, микро. Для использования метрик, описанных в разделе для бинарной классификации, для задач множественной классификации достаточно поменять параметр 'binary' на 'micro' или 'macro'.

Макро – вычисление метрики для каждого класса и получение невзвешенного среднего.

Микро – расчет метрики на глобальном уровне путем подсчета итоговых истинных положительных результатов, ложноотрицательных и ложноположительных результатов (независимо от классов).

Хотя каждый метод усреднения имеет свои преимущества, одним из общих соображений при выборе соответствующего метода является дисбаланс классов. Если у классов разное количество выборок, более информативным может быть использование макро-усреднения, при котором классам меньшинства и большинства назначается одинаковый вес.

Если задача классификации на K классов ставится как K задач об отделении класса i от остальных ($i=1, \dots, K$), то для каждой из них можно посчитать свою матрицу ошибок. Можно использовать следующее усреднение:

$$\begin{aligned}Average Recall &= \frac{\sum_{i=1}^K Recall}{K} \\Average Precision &= \frac{\sum_{i=1}^K Precision}{K} \\Average Accuracy &= \frac{\sum_{i=1}^K Accuracy}{K} \\Average F_1 &= \frac{\sum_{i=1}^K F_1}{K}\end{aligned}\tag{15}$$

3.5. Примеры практического применения с использованием библиотеки **scikit-learn**

Пример 1

Решить задачу бинарной классификации (выжил пассажир или нет). В файле `titanic.csv`²⁴ есть набор данных из 891 строки, включающий в себя: id пассажира, класс каюты, имя, возраст, пол, и самое важное – выжил ли пассажир Титаника или нет. Цель – найти показатели качества для задачи бинарной классификации (выжил пассажир или нет), сделать выводы. С кодом можно ознакомиться по ссылке `titanic.ipynb`²⁵.

Опишем решение задачи бинарной классификации.

1. Импортируем библиотеки

```
import pandas as pd
```

²⁴ <https://drive.google.com/file/d/1YlFndB4HtoRoidGNyO9kmtrIfyLnHuXX/view?usp=sharing>

²⁵ <https://drive.google.com/file/d/1yWh-OHgGsClzQxgqfCqRBoYlKwBzX3oS/view?usp=sharing>

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn import svm
from sklearn.metrics import accuracy_score, precision_score,
recall_score, roc_auc_score
```

2. Загружаем файл с данными и выводим таблицу.

```
data = pd.read_csv('/content/drive/MyDrive/titanic.csv')
data
```

3. Определяются колонки данных, которые будут использоваться как целевая переменная y и влияющие на нее признаки X .

```
columns_target = ['Survived'] # целевая колонка
columns_train = ['Pclass', 'Sex', 'Age', 'Fare'] # колонки, которые
влияли на спасение
```

Мы будем определять, выжил ли пассажир, поэтому нашей целевой колонкой будет Survived (1 или 0, выжил или нет пассажир).

За признаки взяты: класс каюты, пол, возраст и цена билета.

4. Создается матрица переменных и вектор целевой функции

```
X = data[columns_train]
Y = data[columns_target]
```

5. Перед работой с данным нужно проверить, есть ли среди них пропущенные. Для этого стоит воспользоваться функцией `X[column_name].isnull().sum()`, где первая часть возвращает True (пропуск = 1) или False (заполнено = 0), а вторая – суммирует.

Итого мы имеем 177 пропущенных ячеек возраста. Если удалить такое количество данных (а это почти пятая часть от всего), то получим не самые достоверные данные.

```
X['Sex'].isnull().sum()
```

```
np.int64(0)
```

```
X['Pclass'].isnull().sum()
```

```
np.int64(0)
```

```
X['Fare'].isnull().sum()
```

```
np.int64(0)
```

```
X['Age'].isnull().sum()
```

```
np.int64(177)
```

6. Заменим пропущенные данные о возрасте пассажира на среднее значение.

```
X['Age'] = X['Age'].replace({np.nan:X['Age'].mean()})
```

Теперь пустые значения отсутствуют.

7. Для корректной работы классификатору необходимы числовые данные, а не категориальные данные. Поэтому нужно преобразовать данные из колонки пола: male в 0, а female в 1.

```
d={'male':0, 'female':1} # создаем словарь
X['Sex'] = X['Sex'].apply(lambda x:d[x])
```

Замена происходит через функцию lambda, которая принимает значение x и преобразует его с помощью ранее созданного словаря d .

Для проверки выведем столбец пола, все значения заменены на 0 или 1.

```
X['Sex'].head()
```

	Sex
0	0
1	1
2	1
3	1
4	0

dtype: int64

8. Разделим выборку на тестовую и обучающую

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,  
test_size=0.33, random_state=2)
```

Мы используем функцию `train_test_split()` из модуля `sklearn.model_selection` для разделения на обучающую и тестовую выборки, указывая, что размер тестовой – $\frac{1}{3}$ от общего размера данных, а `random_state=2` гарантирует воспроизводимость разделения.

9. Ранее мы уже загрузили модель опорных векторов

```
predmodel = svm.LinearSVC()
```

Здесь создается объект модели машинного обучения методом опорных векторов (SVM). В данном случае используется линейный SVM, коэффициенты которого оптимизируются методом наименьших квадратов.

10. Обучим нашу модель на обучающих данных `X_train, Y_train`

```
predmodel.fit(X_train, Y_train)
```

Метод `fit()` обучает модель на представленных данных.

11. Через метод `predict()` проверим предсказания для тестовой выборки для 10 новых данных.

```
predmodel.predict(X_test[0:10])
```

12. Проверим точность предсказания, используя метрики качества: accuracy, precision, recall, AUC.

```
accuracy = accuracy_score(Y_test, predmodel.predict(X_test))  
precision = precision_score(Y_test, predmodel.predict(X_test))  
recall = recall_score(Y_test, predmodel.predict(X_test))  
F1 = 2 * (precision * recall) / (precision + recall)  
auc = metrics.roc_auc_score(Y_test, predmodel.predict(X_test))
```

Здесь все метрики, кроме F1, обрабатываются через свою функцию, которая сравнивает метки, полученные `predmodel.predict(X_test)` с фактическими метками класса `Y_test`. F1 считается по формуле среднего гармонического precision и recall.

В конце выводим значения всех метрик:

```
print("Метрики качества:")  
print("Accuracy:", accuracy)  
print("Precision:", precision)  
print("Recall:", recall)  
print("F-1 мepa:", F1)
```

```
print("AUC:", auc)
```

Вывод:

Метрики качества:

Accuracy: 0.688135593220339

Precision: 0.5760869565217391

Recall: 0.8833333333333333

F-1 мера: 0.6973684210526314

AUC: 0.7188095238095238

Исходя из полученных значений:

Модель правильно классифицирует 68.81% образцов тестовых данных.

Из всех положительно предсказанных классов, 57.61% являются правильными.

Модель правильно классифицирует 88.33% всех положительных классов.

Площадь под кривой ROC составляет 0.718, что выше 0.5 – значит, значения не случайны.

Для улучшения показателей качества можно взять больше признаков X , а также применить другие классификаторы для решения задачи бинарной классификации.

Пример 2

Рассмотрим задачу бинарной классификации – определение сердечных заболеваний в зависимости от различных показателей (возраста, пола, веса, роста, уровня холестерина, курения и т.п.). С набором данных можно ознакомиться по ссылке [cardio_binary.csv](#)²⁶, с полным кодом можно ознакомиться по ссылке [cardio_binary.ipynb](#)²⁷.

Рассмотрим основные блоки решения задачи.

1. Подключение стандартных библиотек.

```
import pandas as pd
import numpy as np
import gdown
import os
from PIL import Image
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import accuracy_score, roc_auc_score, f1_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
from lightautoml.automl.presets.tabular_presets import TabularAutoML
from lightautoml.tasks import Task
from sklearn.preprocessing import StandardScaler
```

2. Загрузка данных и разделение их на множества *train* и *test*.

²⁶ <https://docs.google.com/spreadsheets/d/1lfiFYkaUhHPOv7EzkNk-eNJBleHBaw5yzcyKWg1X5o0/edit?usp=sharing>

²⁷ https://drive.google.com/file/d/1q79MLk7DS7pmPP5tGX5ui_HEXWinrMML/view?usp=sharing

```
path = "/content/gdrive/MyDrive/Colab Notebooks/ML/ML/cardio_binary.csv"
```

```
data = pd.read_csv(path, delimiter=';')
X = data.drop(columns=["cardio", "id"]) # Удаляем целевой столбец и 'id'
y = data["cardio"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
```

3. Масштабирование исходных данных

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

4. Задаем показатели качества для задачи бинарной классификации (метрики Accuracy, Roc Auc, F1).

```
models_metrics = {}
def calc_metrics(true, pred):
    stats={
        'Accuracy': round(metrics.accuracy_score(true, pred),4),
        'F1': round(metrics.f1_score(true, pred),4),
        'ROC AUC': round(metrics.roc_auc_score(true, pred),4)
    }
    print(f"Accuracy: {stats['Accuracy']}\nF1: {stats['F1']}\nROC AUC: {stats['ROC AUC']}")
    return stats
```

5. Определяем модели как DecisionTreeClassifier(), RandomForestClassifier(), SVC(), LogisticRegression() и с помощью LAMA. Обучаем модель на данных train (метод fit()). Предсказываем метки классов (0 или 1) для множества test. Выводим показатели точности. Для наглядности сводим все данные в одну таблицу.

```
model_cls_compare = pd.DataFrame(models_metrics).T
model_cls_compare
```

	Accuracy	F1	ROC AUC
decision_tree	0.6359	0.6361	0.6360
RandomForestClassifier	0.7170	0.7129	0.7174
SVC	0.7245	0.7159	0.7251
LogisticRegression	0.7185	0.7066	0.7192
LAMA	0.7390	0.7282	0.7390

Вывод: для данной задачи среди пяти известных классификаторов наилучшие результаты показал фреймворк LAMA.

Пример 3

Рассмотрим задачу многоклассовой классификации – распознавание видов рыб в зависимости от их характеристик (вес, длина и т.п). Задача состоит в том, чтобы по характеристикам определить один из семи видов рыб. С набором данных можно ознакомиться по ссылке `fish.csv`²⁸, с кодом можно ознакомиться по ссылке `fish_multy_cl.ipynb`²⁹.

Датасет представляет собой табличные данные. Несколько первых строк из датасета представлены на рисунке ниже.

Species	Weight	Length1	Length2	Length3	Height	Width
Bream	242	23.2	25.4	30	11.52	4.02
Bream	290	24	26.3	31.2	12.48	4.3056
Bream	340	23.9	26.5	31.1	12.3778	4.6961
Bream	363	26.3	29	33.5	12.73	4.4555
Bream	430	26.5	29	34	12.444	5.134
Bream	450	26.8	29.7	34.7	13.6024	4.9274
Bream	500	26.8	29.7	34.5	14.1795	5.2785
Bream	390	27.6	30	35	12.67	4.69

Рис. 14. Первые строки датасета `fish.csv`

Опишем основные блоки.

1. Загружаем данные

```
# Загрузка данных
df = pd.read_csv('/content/gdrive/MyDrive/Colab Notebooks/ML/ML/fish.csv', delimiter=',')

# Разделение на признаки (X) и целевую переменную (y)
X = df.loc[:, df.columns != 'Species']
y = df['Species']
```

2. Выводим уникальные значения, т.е. классы

```
import numpy as np
print(np.unique(y))

['Bream' 'Parkki' 'Perch' 'Pike' 'Roach' 'Smelt' 'Whitefish']
```

3. Ставим каждому классу в соответствие метку (от 0 до 6, т.к. у нас 7-мь классов).

²⁸ <https://drive.google.com/file/d/1KdQ2rxHUYbPnlaliiKW78Te5JhumAb7B/view?usp=sharing>

²⁹ <https://drive.google.com/file/d/1TZttNFImxsAMi8DxAT3-cE1s8qclSjgN/view?usp=sharing>

```

Y=[]
for i in y:
    if i.find("Bream") != -1:
        Y.append(0)
    elif i.find("Parkki") != -1:
        Y.append(1)
    elif i.find("Perch") != -1:
        Y.append(2)
    elif i.find("Roach") != -1:
        Y.append(3)
    elif i.find("Smelt") != -1:
        Y.append(4)
    else:
        Y.append(5)
print((Y))
Y = np.array(Y)

```

4. Разбиваем множество на train и test

```

X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.3, random_state=12)

```

5. Определяем метрики качества. Обратим внимание на то, что для задачи многоклассовой классификации необходимо провести усреднение по всем меткам (average='macro').

```

from sklearn import metrics
def calc_metrics_cls(true, pred):
    stats = {
        'Accuracy': metrics.accuracy_score(true, pred),
        'F1': metrics.f1_score(true, pred, average='macro')
    }
    print(f"Accuracy: {stats['Accuracy']}\nF1: {stats['F1']}")
    return stats
cls_models_stats = {}

```

6. Определяем модели DecisionTreeClassifier() и RandomForestClassifier(), обучаем модели (fit()), предсказываем метки классов для множества test (predict()), выводим метрики качества.

```
cls_model_dt = DecisionTreeClassifier()
cls_model_dt.fit(X_train, Y_train)
y_pred_cls_dt = cls_model_dt.predict(X_test)
cls_models_stats['DecisionTree'] = calc_metrics_cls(Y_test, y_pred_cls_dt)
```

Accuracy: 0.6875
F1: 0.5897220701818403

```
cls_model_rf = RandomForestClassifier()
cls_model_rf.fit(X_train, Y_train)
y_pred_cls_rf = cls_model_rf.predict(X_test)
cls_models_stats['RandomForest'] = calc_metrics_cls(Y_test, y_pred_cls_rf)
```

Accuracy: 0.7708333333333334
F1: 0.7018278018278018

```
model_cls_compare = pd.DataFrame(cls_models_stats).T
model_cls_compare
```

	Accuracy	F1
RandomForest	0.770833	0.701828
DecisionTree	0.687500	0.589722

Опишем общий алгоритм решения задачи классификации.

1. Считываем данные. Формируем признаки X и целевую переменную Y . Категориальные данные переводим в числовые (например, с помощью `LabelEncoder()` или `OneHotEncoder`), обрабатываем пропуски в данных (например, удаляя строки с пропусками или заменяя их средним значением).
2. Разбиваем исходное множество на train и test (`train_test_split()`). На множестве train будем обучать модель, на множестве test будем считать показатели качества.
3. Определяем модель (`LogisticRegression()`, `SVC()`, `LAMA` и т.п.).
4. Обучаем модель (`fit()`) на множествах X_{train} , Y_{train} .
5. Предсказываем (`predict()`) значения целевой переменной на множестве X_{test} . Получаем значения Y_{predict} , полученные моделью.
8. Вычисляем показатели качества (Accuracy, F1) для Y_{predict} и Y_{test} .
9. Делаем выводы о том, какая модель для решения задачи классификации подходит лучше (лучше та модель, у которой выше показатели качества, например, Accuracy, F1).

3.6. Выводы

В данном разделе была рассмотрена задача классификации и методы ее решения (например, логистическая регрессия, деревья решений и метод опорных векторов).

Классификация – это тип задачи обучения с учителем, где модель учится присваивать объектам одну из заранее заданных категорий (классов) на основе их признаков.

В разделе были введены показатели качества задачи классификации (Accuracy, F1, ROC-AUC). Также были рассмотрены примеры, демонстрирующие применения описанных подходов на практике, с полным кодом всех примеров можно ознакомиться по ссылкам в разделе или по QR в заключении.

Этот раздел может быть полезен при решении задач классификации, например, при определении по сердечно-сосудистых заболеваний, а также при подготовке к независимому экзамену по анализу данных.

4. Задача кластеризации

4.1. Постановка задачи

Кластеризация – это метод группировки данных. Предположительно, данные, присутствующие в одной группе, обладают схожими характеристиками или свойствами. С другой стороны, данные, входящие в отдельные группы (кластеры), обладают уникальными характеристиками или свойствами. Задача кластеризации - группировка объектов на основе их сходства, поиск структуры в данных без явного задания целевых переменных. В результате объекты, наиболее похожие друг на друга, объединяются в один кластер. Задача кластеризации относится к методу обучения без учителя, т.е. для данных нет меток классов [4].

Примеры кластеризации: разделение клиентов интернет-магазина по их предпочтениям, группировка новостей по схожести контента, группировка стран по уровню экономического развития.

Задаче кластеризации в общем виде можно описать следующим образом:

Пусть X – множество объектов, Y - множество идентификаторов (меток) кластеров, где кластер – некое количество данных, объединенных вместе на основе какого-либо признака. На множестве X задана функция расстояния между объектами $\rho(x, x')$. Дана конечная обучающая выборка объектов $X_m = \{x_1, \dots, x_m\} \subset X$. Необходимо разбить выборку на подмножества (кластеры), то есть каждому объекту $x_i \in X_m$ сопоставить метку $y_i \in Y$, где i определено $[1, m]$, таким образом, чтобы объекты внутри каждого кластера были близки относительно метрики ρ , т.е. имели малое расстояние друг от друга, а объекты из разных кластеров значительно различались, то есть расстояние от объекта одного класса до объекта другого класса было бы большим [7].

4.2. Методы решения задачи кластеризации

- **Метод К-средних или kmeans.**

Цель метода – разделение m наблюдений на k кластеров, при этом каждое наблюдение относится к тому кластеру, к центру (центроиду) которого оно ближе всего. Алгоритм формирования кластеров:

1. Выбрать количество кластеров k , которое кажется оптимальным.
2. Случайным образом из данных выбирать k точек (центроидов).
3. Для каждой точки рассчитать ближайший центроид (расстояние Евклида). Сформировать кластеры.
4. Каждый центроид каждого i -го кластера пересчитать через среднее значение всех точек данных, принадлежащих к данному кластеру по формуле:

$$\mu_j = \frac{1}{S_j} \sum_{x^{(j)} \in S_j} x^{(j)}. \quad (16)$$

5. Последние два шага повторяются до тех пор, пока центры кластеров не перестанут двигаться или количество итераций не превысит заранее заданное небольшое значения.

Пример алгоритма приведен на рисунке 14.

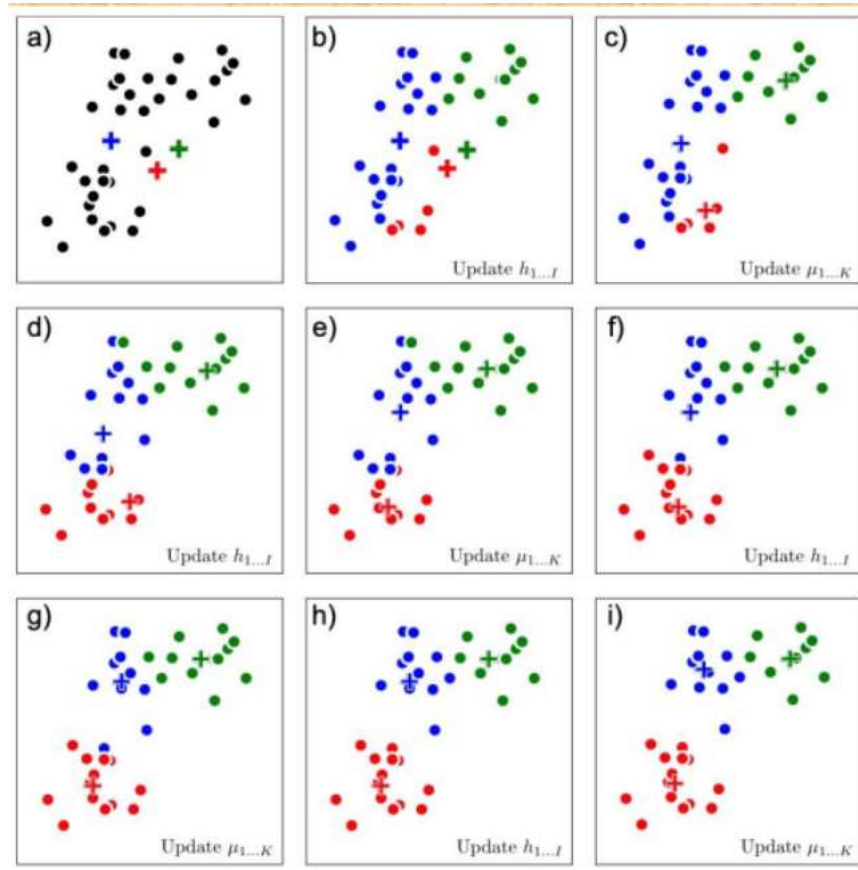


Рис.14. Демонстрация работы метода kmeans при разбиении данных на 3 кластера: а) Выбор случайным образом 3-х центроидов – шаг 2 алгоритма; б) каждой точке ставится в соответствие ближайший центроид – шаг 3; в) пересчитываем центроид каждого класса – шаг 4; д) каждой точке ставится в соответствие ближайший центроид – шаг 3; е) пересчитываем центроид каждого класса – шаг 4; ф) каждой точке ставится в соответствие ближайший центроид – шаг 3; г) пересчитываем центроид каждого класса – шаг 4; h) каждой точке ставится в соответствие ближайший центроид – шаг 3; и) на этом шаге центры кластеров не изменяются, поэтому процесс разбиения данных на кластеры завершен

- **Метод DBScan** (Density-based spatial clustering of applications with noise, плотностной алгоритм пространственной кластеризации с присутствием шума).

На вход принимает матрицу близости и два параметра – радиус окрестности и количество соседей.

Идея заключается в выявлении кластеров путем поиска областей с высокой плотностью точек внутри них. На вход принимается матрица близости и два параметра – радиус окрестности и количество соседей, затем выбирается не рассмотренная точка, если

в заданном ранее радиусе есть заданное количество соседей, то точка объединяется с ними в кластер, в ином случае идентифицируется как шум. Алгоритм выполняется рекурсивно для каждой точки данных в кластере, в результате присоединяя туда другие точки, находящиеся в заданном радиусе. Это повторяется для всех не рассмотренных ранее точек. Плюсом DBScan является возможность создания кластеров любой формы. Пример метода представлен на рисунке ниже.

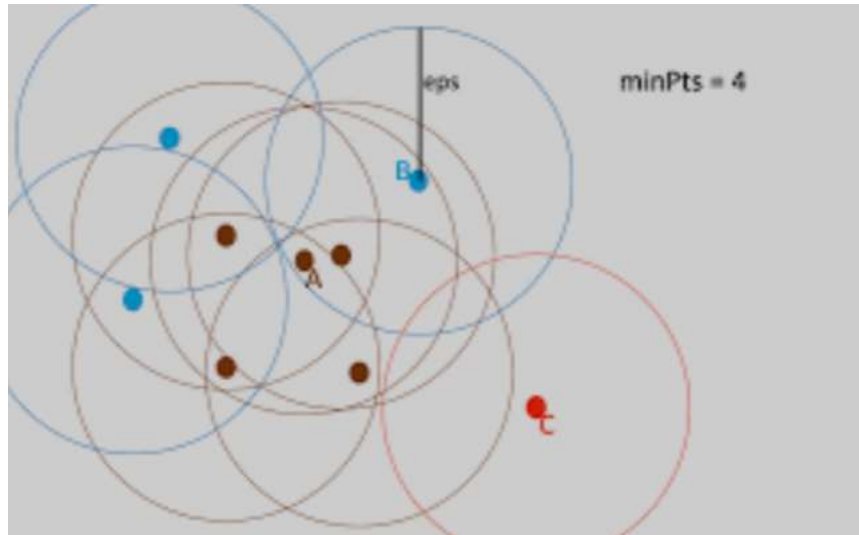


Рис.15. Пример работы метода DBScan, радиус окрестности равен ϵ , количество соседей 4

- **HDBScan** (Hierarchical Density-Based Spatial Clustering of Applications with Noise, иерархический плотностной алгоритм пространственной кластеризации с присутствием шума)

Это более развитый алгоритм DBScan, который позволяет иерархически строить кластеры и добавляет опцию автоматического определения оптимального количества кластеров на основе данных о плотности. Главным преимуществом перед DBScan является возможность работы с неоднородными данными и обработке шума (точек, оказавшихся вне кластера). Алгоритм начинает работать с дендрограммы – дерева, построенное по матрице мер близости между кластерами, затем выбирается оптимальное разбиение дерева на кластеры путем поиска наиболее плотных областей и объединения точек данных в соответствующие кластеры.

4.3. Метрики качества

Метрика силуэт. Позволяет оценить качество кластеризации, используя только саму (неразмеченную) выборку и результат кластеризации. Сначала силуэт определяется отдельно для каждого объекта. Обозначим через a – среднее расстояние от данного объекта

до объектов из того же кластера, через b – среднее расстояние от данного объекта до объектов из ближайшего кластера (отличного от того, в котором лежит сам объект). Тогда силуэтом данного объекта называется величина, равная отношению $(b-a)$ и $\max(a, b)$.

$$s = \frac{b - a}{\max(a, b)} \quad (17)$$

Таким образом, силуэт показывает, насколько среднее расстояние до объектов своего кластера отличается от среднего расстояния до объектов других кластеров. Величина лежит в диапазоне $[-1, 1]$. Если кластеризация плохая (кластеры разрозненные), то ее значение будет близко к -1 ; если в районе 0 , то кластеры пересекаются и накладываются друг на друга; у «плотных» четко выделенных кластеров значения силуэта близко к 1 . Чем больше силуэт, тем более четко выделены кластеры, и они представляют собой компактные, плотно сгруппированные облака точек.

Две следующие метрики (гомогенность и полнота) определяются с использованием функций энтропии и условной энтропии. Энтропия – это мера неопределенности или "беспорядка" в системе.

Метрика гомогенности используется для оценки, насколько объекты внутри одного кластера схожи или однородны друг с другом.

Индекс гомогенности рассчитывается по формуле:

$$h = 1 - \frac{H(C|K)}{H(C)} \quad (18)$$

где K – результат кластеризации, а C – истинное разбиение выборки на классы, H – энтропия.

Величина принимает значения в диапазоне $[0, 1]$, и большие значения соответствуют более точной кластеризации. Лучший способ максимизировать индекс гомогенности – выделить каждый объект выборки в отдельный класс.

Метрика полноты используется для оценки того, насколько объекты одного класса относятся к одному классу. Рассчитывается по формуле:

$$c = 1 - \frac{H(K|C)}{H(K)} \quad (19)$$

Точно так же, как и в гомогенности, индекс полноты лежит в диапазоне от 0 до 1 , и большие значения соответствуют более точной кластеризации. Однако в случае с полнотой, лучший способ максимизировать значение – объединить все данные в один класс.

V-мера. Метрика имеет аналогичный смысл с F1-мерой из классификации: среднее гармоническое между полнотой и гомогенностью. Вычисляется по формуле:

$$v = 2 \cdot \frac{h \cdot c}{h + c} \quad (20)$$

V-мера принимает значения от 0 до 1, где 1 означает идеальную кластеризацию, а 0 – случайное распределение элементов по кластерам.

4.4. Примеры практического применения

Пример 1

Рассмотрим датасет Country-data.csv.csv³⁰ с информацией о странах, включающий экономические показатели (смертность, показатели импорта, экспорта, инфляции, доход населения, продолжительность жизни и т.п.). Цель – провести кластеризацию (т.е. выделить кластеры со странами) методами K-means, DBScan, HDBScan, найти показатели качества. С полным кодом можно ознакомиться по ссылке country_cl.ipynb³¹.

Опишем основные шаги:

1. Импортируем необходимые библиотеки

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.metrics import silhouette_score, homogeneity_completeness_v_measure
```

2. Загружаем данные и выводим первые пять строк. Для каждой страны есть признаки (уровень дохода (income), ВВП (gdpp), уровень здоровья (health) и т.п.).

```
# загрузка данных
data = pd.read_csv('/content/gdrive/MyDrive/Colab Notebooks/ML/ML/Country-data.csv', sep=',')
data.head(5)
```

	country	child_mort	exports	health	imports	income	inflation	life_expec	total_fer	gdpp
0	Afghanistan	90.2	10.0	7.58	44.9	1610	9.44	56.2	5.82	553
1	Albania	16.6	28.0	6.55	48.6	9930	4.49	76.3	1.65	4090
2	Algeria	27.3	38.4	4.17	31.4	12900	16.10	76.5	2.89	4460
3	Angola	119.0	62.3	2.85	42.9	5900	22.40	60.1	6.16	3530
4	Antigua and Barbuda	10.3	45.5	6.03	58.9	19100	1.44	76.8	2.13	12200

3. Обработка данных

³⁰ <https://drive.google.com/file/d/1aNnKtiEikq037x6t7qmreSJOt5dhsIB-/view?usp=sharing>

³¹ <https://drive.google.com/file/d/1MY8IZpoqFiRaL6A3XZl-W2V2tY80tR8U/view?usp=sharing>

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import homogeneity_score, completeness_score, silhouette_score
```

```
countries = df['country']
X = df.drop(columns=['country'])
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

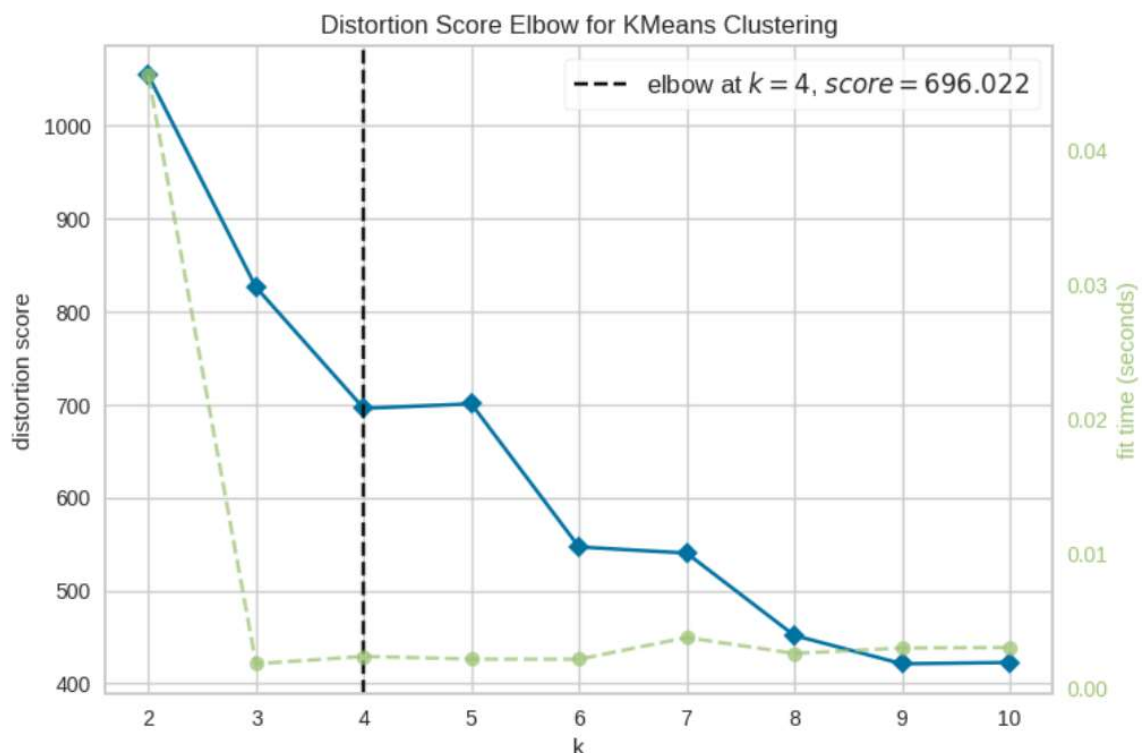
4. Воспользуемся методом локтя, чтобы найти оптимальное значение кластеров для метода kmeans. Для этого построим график зависимости WCSS (within-cluster sum of squares, сумма квадратов внутрикластерных расстояний до центра кластера) от количества кластеров (k):

$$WCSS = \sum_{j=1}^k \sum_{x_i \in C_j} \|x_i - c_j\|^2 \quad (21),$$

где k – число кластеров, C_j – j -й кластер, x_i – i -ая точка данных в кластере, c_j – центроид j -го кластера, $\|x_i - c_j\|^2$ – квадрат евклидова расстояния от точки до центроида.

```
# Использование метода локтя для установления оптимального количества кластеров
print('Elbow Method to determine the number of clusters to be formed:')
Elbow_M = KElbowVisualizer(KMeans(), k=10) # визуализирование метода локтя
Elbow_M.fit(X_scaled) # обучение модели на данных
Elbow_M.show() # отображение
```

... Elbow Method to determine the number of clusters to be formed:



На графике видно, что $k=4$, т.е. при этом значении дальнейшее увеличение числа кластеров не приводит к значительному снижению внутриклассовой дисперсии.

5. Установим параметры метода kmeans и проведем кластеризацию:

```
kmeans = KMeans(n_clusters=4, init='k-means++', n_init=50, random_state=3)
km_labels = kmeans.fit_predict(X_scaled)
```

Здесь число кластеров равно 4 согласно методу локтя, `init='k-means++'`, инициализирует центроиды так, чтобы они были удалены друг от друга, что может дать лучшие результаты, чем случайная инициализация, `n_init=50` попытает разные варианты размещения начальных точек, запустив k-means 50 раз, и выберет тот вариант, который минимизирует сумму квадратичных расстояний внутри кластера.

6. Выведем метрики качества для kmeans, dbscan, hdbscan

```
print("KMeans")
print("KMeans silhouette:", silhouette_score(X_scaled, km_labels))
print("KMeans homogeneity:", homogeneity_score(true_labels, km_labels))
print("KMeans completeness:", completeness_score(true_labels, km_labels))
```

```
from sklearn.cluster import DBSCAN
from sklearn.neighbors import NearestNeighbors
from sklearn.preprocessing import StandardScaler

dbscan = DBSCAN(eps=1.2, min_samples=3, metric='euclidean') # Установка максимального расстояния, минимального количества точек
dbscan_labels = dbscan.fit_predict(X_scaled) # К каждой точке в данных присваивается метка кластера
# Проверка уникальных меток DBScan

unique_labels = set(dbscan_labels)
print("Уникальные метки DBScan:", unique_labels) # Вывод уникальных меток, где -1 является выбросом

# Проверка, если все метки -1, попробуйте изменить параметры

if len(unique_labels) == 1 and -1 in unique_labels:
    print("DBScan не обнаружил кластеры. Попробуйте настроить параметры.")
else:
    # Оценка качества DBScan
    print("DBScan silhouette:", silhouette_score(X_scaled, dbscan_labels))

h_score = homogeneity_score(true_labels, dbscan_labels)
c_score = completeness_score(true_labels, dbscan_labels)

print("DBScan Homogeneity:", h_score)
print("DBScan completeness:", c_score)
```

```
hdbscan_clusterer = hdbscan.HDBSCAN(min_cluster_size=3, min_samples=2)

hdbscan_labels = hdbscan_clusterer.fit_predict(X_scaled)
h_score = homogeneity_score(true_labels, hdbscan_labels)
c_score = completeness_score(true_labels, hdbscan_labels)

print("HDBScan Silhouette Score:", silhouette_score(X_scaled, hdbscan_labels))
print("HDBSCAN Homogeneity Score:", h_score)
print("HDBSCAN Completeness Score:", c_score)
```

Метрики качества для этого набора данных получились лучше у метода kmeans и составили:

```
KMeans  
KMeans silhouette: 0.30001028829630455  
KMeans homogeneity: 0.4172600627100487  
KMeans completeness: 0.2982093038862092
```

4.5. Выводы

В данном разделе была рассмотрена задача кластеризации и методы ее решения (KMeans, DBScan, HDBScan).

Кластеризация – это метод машинного обучения без учителя, при котором алгоритм автоматически группирует объекты данных в кластеры на основе их сходства, без использования заранее известных меток

В разделе были введены показатели качества задачи кластеризации (метрики силуэт, гомогенность, полнота). Также были рассмотрены примеры, демонстрирующие применения описанных подходов на практике, с полным кодом всех примеров можно ознакомиться по ссылках в разделе или по QR коду в заключении.

Заключение

В учебном пособии описаны шаги для подбора подходящих моделей при решении задач машинного обучения для табличных данных (линейная регрессия, классификация, кластеризация) и оценка моделей на основе корректных показателей качества. Рассмотрены методы решения задач машинного обучения: стандартные классификаторы из библиотеки scikit-learn, а также фреймворк LAMA. В пособии описаны алгоритмы для решения задач в области машинного обучения, например, предсказание суммы страховых выплат, распознавание болезней, распределение стран по уровню экономического развития и т.п., даны практические рекомендации, особый акцент сделан на анализе практических примеров. Все практические примеры и датасеты, описанные в пособии можно посмотреть в каталоге по ссылке

https://drive.google.com/drive/folders/1o6jXIDvyhrfLw3BfqmbbRE1wAOxb3usB?usp=drive_link или QR коду:



Следуя рекомендациям, представленным в данном учебном пособии, студенты смогут решить задачи в области машинного обучения и оценить качество выбранной модели.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Murphy K. P. Probabilistic machine learning: Advanced topics. – MIT press, 2023.
2. Badillo S. et al. An introduction to machine learning //Clinical pharmacology & therapeutics. – 2020. – Т. 107. – №. 4. – С. 871-885.
3. Janiesch C., Zszech P., Heinrich K. Machine learning and deep learning //Electronic Markets. – 2021. – Т. 31. – №. 3. – С. 685-695.
4. Sharifani K., Amini M. Machine learning and deep learning: A review of methods and applications //World Information Technology and Engineering Journal. – 2023. – Т. 10. – №. 07. – С. 3897-3904.
5. Shafiq M., Gu Z. Deep residual learning for image recognition: A survey //Applied sciences. – 2022. – Т. 12. – №. 18. – С. 8972.
6. Bird J. J., Lotfi A. Cifake: Image classification and explainable identification of ai-generated synthetic images //IEEE Access. – 2024. – Т. 12. – С. 15642-15650.
7. Ahlawat H., Aggarwal N., Gupta D. Automatic Speech Recognition: A survey of deep learning techniques and approaches //International Journal of Cognitive Computing in Engineering. – 2025. – Т. 6. – С. 201-237.
8. Prabhavalkar R. et al. End-to-end speech recognition: A survey //IEEE/ACM Transactions on Audio, Speech, and Language Processing. – 2023. – Т. 32. – С. 325-351.
9. Khan W. et al. Exploring the frontiers of deep learning and natural language processing: A comprehensive overview of key challenges and emerging trends //Natural Language Processing Journal. – 2023. – Т. 4. – С. 100026.
10. Fanni S. C. et al. Natural language processing //Introduction to artificial intelligence. – Cham : Springer International Publishing, 2023. – С. 87-99.
11. Zhou B. et al. Natural language processing for smart healthcare //IEEE Reviews in Biomedical Engineering. – 2022. – Т. 17. – С. 4-18.
12. Бурков А. Машинное обучение без лишних слов. – " Издательский дом"" Питер""", 2020.
13. Raschka S. et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. – Packt Publishing Ltd, 2022.
14. James G. et al. An introduction to statistical learning: With applications in python. – Springer Nature, 2023.
15. Savchenko A. V., Savchenko L. V., Makarov I. Fast search of face recognition model for a mobile device based on neural architecture comparator //IEEE Access. – 2023. – Т. 11. – С. 65977-65990.
16. Savchenko A. V. AutoFace: How to Obtain Mobile Neural Network-Based Facial Feature Extractor in Less Than 10 Minutes? //IEEE Access. – 2024. – Т. 12. – С. 25106-25118.
17. Churaev E., Savchenko A. V. A standalone software for real-time facial analysis in online conferences and e-lessons //Software Impacts. – 2023. – Т. 16. – С. 100507.
18. Шолле Ф. Глубокое обучение на Python. 2-е межд. издание. – Питер, 2023.
19. Sushma S. A., TG K. K. Comparative study of naive bayes, gaussian naive bayes classifier and decision tree algorithms for prediction of heart diseases //International Journal for Research in Applied Science and Engineering Technology. – 2021. – Т. 9. – №. 3. – С. 475-486.
20. Natekin A., Knoll A. Gradient boosting machines, a tutorial //Frontiers in neurorobotics. – 2013. – Т. 7. – С. 63623.

Приложение А. Задания для самостоятельного решения

1. Для задачи предсказания цены автомобиля в зависимости от его характеристик найти не менее трех показателей качества. Использовать стандартные классификаторы и LAMA.

Датасет:

<https://drive.google.com/file/d/1Eg378VHViRbRdsbcW6od3IjUPtkZTizG/view?usp=sharing>

2. Определить будет ли дождь (RainTomorrow) в зависимости от характеристик погоды (ветра, облаков и т.п.). Найти не менее трех показателей качества. Использовать стандартные классификаторы (не менее пяти) и LAMA. Добиться значения показателя Accuracy не менее 80%.

Датасет:

https://drive.google.com/file/d/16EKUua6hEb7_Pd03jLsyrIBsXEXATKVF/view?usp=sharing

3. Для задачи предсказания цены автомобиля в зависимости от его характеристик найти не менее трех показателей качества. Использовать стандартные классификаторы и LAMA.

Датасет:

https://drive.google.com/file/d/1Xfj0n07Ln6tHpC0nA_QMHUxN4HI7zsdq/view?usp=sharing

4. Определить уровень дохода (income) в зависимости от характеристик (пол, образование, семейное положение и т.п.). Найти не менее трех показателей качества. Использовать стандартные классификаторы, LAMA. Провести сравнение классификаторов и LAMы по трем показателям качества. Добиться значения показателя Accuracy не менее 80%.

Датасет:

<https://drive.google.com/file/d/1vWgTPhd3vOwvgyM7KpJ5zsZROONEUL5s/view?usp=sharing>

5. Определить уровень здоровья (fetal_health). Найти не менее двух показателей качества. Использовать стандартные классификаторы и LAMA. Провести сравнение классификаторов и LAMы по показателям качества. Сделать выводы. Добиться значения показателя Accuracy не менее 90%.

Датасет:

https://drive.google.com/file/d/1kR86usoOZQc7TJYO_-sX7xOzJAKF9oji/view?usp=sharing

6. Решить задачу кластеризации с помощью методов kmeans, DBSCAN, hdbscan, найти показатель качества силуэт.

Для каждой группы вывести количество элементов и усредненное количество (Mean) в группе, построить дендрограмму.

Датасет:

<https://drive.google.com/file/d/1IziOo81EPwNdj5Fj5vlk3D9hBdBS9x0n/view?usp=sharing>

7. Решить задачу кластеризации с помощью методов kmeans, DBSCAN, hdbscan, найти показатель качества силуэт.

Для каждой группы вывести количество элементов и усредненное количество (Animal fats) в группе, построить дендрограмму.

Датасет:

<https://drive.google.com/file/d/1IziOo81EPwNdj5Fj5vlk3D9hBdBS9x0n/view?usp=sharing>

8. Решить задачу кластеризации с помощью методов kmeans, DBSCAN, hdbscan, найти показатель качества силуэт. Для каждой группы вывести количество элементов и средний показатель TotalCharges.

Датасет:

https://drive.google.com/file/d/1IVNSYqzGjSycN_tgDr4TRNrr9Hk7WhXF/view?usp=sharing

Л.В. Савченко, И.А. Макаров, А.В. Савченко

**Машинное обучение
и анализ табличных данных**

Учебно-методическое пособие

ISBN 978-5-907929-35-7



9 785907 929357 >

Подписано в печать 18.06.2026
Бумага офсетная. Формат 60x84/16
Усл. печ. л. 3,43
Тираж 100 экз. Заказ № 6145
Постер-М
119296, Москва, ул. Молодежная, 3